

EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY



TRƯỜNG ĐẠI HỌC
QUỐC TẾ
MIỀN ĐÔNG
EASTERN
INTERNATIONAL
UNIVERSITY

CAPSTONE PROJECT 1
OFFICE HOURS

Tran Binh Trong – 2231200121

Chau Nhat Tang – 2231200117

Nguyen Dinh Thien – 2231200118

Thai Phan Thanh Sang – 2231200114

Software Engineering

Ho Chi Minh City, August, 2026

EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY



**TRƯỜNG ĐẠI HỌC
QUỐC TẾ
MIỀN ĐÔNG**

**EASTERN
INTERNATIONAL
UNIVERSITY**

CAPSTONE PROJECT 1
OFFICE HOURS

Tran Binh Trong – 2231200121

Chau Nhat Tang – 2231200117

Nguyen Dinh Thien – 2231200118

Thai Phan Thanh Sang – 2231200114

Software Engineering

Ho Chi Minh City, August, 2026

ABSTRACT

The OfficeHours project was developed to solve common challenges in scheduling and managing academic advisor consultation sessions within university environments. Many higher education institutions struggle with manual scheduling processes and undetected timetable conflicts, leading to operational inefficiencies and inequitable student access. OfficeHours aims to centralize scheduling workflows, automate conflict detection, and implement fair resource allocation to support better academic advising and student success.

The platform offers a range of features including automated conflict prevention, dynamic waitlist management, customizable allocation policies, and real-time status tracking. Whether consultation slots are open, fully booked, or experiencing peak demand, OfficeHours provides an up-to-date view of advisor availability and queuing status. Students can submit consultation requests through the system and track their scheduling progress or waitlist position in a structured way.

OfficeHours is built using a modern and scalable technology stack. The web interface is powered by Next.js and React to ensure a responsive and intuitive user experience. The codebase benefits from TypeScript, which enhances system reliability through static typing. On the backend, PostgreSQL handles structured data storage while employing advanced exclusion constraints with GiST indexing to prevent double-booking and concurrency anomalies under high traffic.

This software solution not only resolves scheduling conflicts and improves operational efficiency but also significantly enhances access equity – reducing the Gini coefficient of inequality from 0.25 to 0.10 through a hybrid allocation mechanism. Future enhancements may include campus learning management system integration, mobile push notifications, and AI-driven analytics for demand forecasting. OfficeHours is a practical and flexible platform designed to adapt to institutional needs, providing long-term value through streamlined workflows and equitable resource distribution.

ACKNOWLEDGEMENT

We would like to take this opportunity to express our sincere gratitude to all those who have supported and contributed to the successful completion of the OfficeHours project. Their guidance, technical insights, and encouragement have been invaluable across every phase of this research.

First and foremost, we extend our heartfelt thanks to the Faculty of Information Technology at Eastern International University (EIU). The faculty's commitment to fostering an academic environment in software engineering, distributed systems, and algorithmic research provided the essential foundation for our project.

Our profound appreciation goes to our supervisor, Mr. Nguyen Tien Tuan Khiem, whose expert guidance and unwavering mentorship were crucial from initial planning to system implementation. Their constructive feedback, patience, and direction helped us overcome complex technical challenges and refine the overall quality of our work.

Lastly, we extend our thanks to the academic advisors, domain experts, and team members whose collaboration and dedication were fundamental to achieving our goals. Their valuable insights during requirement validation and testing created a productive atmosphere that propelled this project forward.

TABLE OF CONTENTS

ABSTRACT	i
ACKNOWLEDGEMENT	ii
TABLE OF CONTENTS	iii
LIST OF FIGURE	vi
LIST OF TABLES.....	viii
LIST OF ABBREVIATIONS	ix
CHAPTER 1. OVERVIEW	1
1.1. Introduction	1
1.2. Project Objectives.....	1
1.3. Challenges	2
1.4. Report Structure.....	3
CHAPTER 2. INTRODUCTION TO TECHNOLOGIES	4
2.1. Next.js and React.....	4
2.1.1. Introduction to Next.js and React.....	4
2.1.2. Advantages of Next.js and React.....	4
2.2. Tailwind CSS.....	4
2.2.1. Introduction to Tailwind CSS.....	4
2.2.2. Advantages of Tailwind CSS	4
2.3. Framer Motion and UI Libraries	5
2.3.1. Introduction to Framer Motion and UI Libraries.....	5
2.3.2. Advantages of UI Libraries	5
2.4. PDF.js	5
2.4.1. Introduction to PDF.js	5
2.4.2. Advantages of PDF.js.....	5
2.5. Java Spring Boot.....	5
2.5.1. Introduction to Java Spring Boot.....	5
2.5.2. Advantages of Java Spring Boot	6
2.6. PostgreSQL.....	6
2.6.1. Introduction to PostgreSQL.....	6
2.6.2. Advantages of PostgreSQL	6
2.7. JSON Web Token.....	6
2.7.1. Introduction to JSON Web Token.....	6
2.7.2. Advantages of JSON Web Token.....	6

2.8. Git and GitHub	7
2.8.1. Introduction to Git and GitHub	7
2.8.2. Advantages of GitHub	7
2.9. System Architecture and Technology Stack.....	7
2.9.1. Front-end	7
2.9.2. Back-end.....	7
CHAPTER 3. APPLICATION ANALYSIS, DESIGN AND IMPLEMENTATION	8
3.1. System Description.....	8
3.2. Use Case Diagram	9
3.2.1. General Use Case Diagram.....	9
3.2.2. For Administrator	10
3.2.3. For Lecturer	11
3.2.4. For Student	12
3.3. Activity Diagram	13
3.3.1. Login Function	13
3.3.2. Create Function	14
3.3.3. Read Function.....	15
3.3.4. Update Function	16
3.3.5. Delete Function	17
3.3.6. Password Recovery	17
3.4. Database.....	19
3.4.1. Entity-Relationship Diagram.....	19
3.4.2. Database Analysis	19
3.5. Implementation.....	23
3.5.1. Implementation of Scrum	23
CHAPTER 4. EXPERIMENT AND DISCUSSION	24
4.1. Installation Environment	24
4.1.1. Hardware Requirements	24
4.1.2. Software Requirements	24
4.2. Results and Discussion	25
4.2.1. Web Interface Implementation	25
4.2.2. Frontend Evaluation and Discussion	36
CHAPTER 5. CONCLUSION AND FUTURE WORKS	38
5.1. Conclusion.....	38
5.2. Future Works	38

5.2.1. Backend API & Persistence Layer Implementation	39
5.2.2. Mobile Application Development	39
5.2.3. Student Information System (SIS) API Integration.....	39
5.2.4. Real-time Communication & Notification Engine.....	39
5.2.5. Predictive Machine Learning for Demand Forecasting.....	39
REFERENCES	40

LIST OF FIGURE

Figure 1. OfficeHours Academic Consultation System Diagram	9
Figure 2. Administrator Subsystem Diagram	10
Figure 3. Lecturer Subsystem Diagram	11
Figure 4. Student Subsystem Diagram	12
Figure 5. Login Function Diagram	13
Figure 6. Create Function Diagram	14
Figure 7. Read Function Diagram	15
Figure 8. Update Function Diagram	16
Figure 9. Delete Function Diagram	17
Figure 10. Password Diagram	18
Figure 11. Entity-Relationship Diagram	19
Figure 12. Trello – Project Management.....	23
Figure 13. Landing Page.....	25
Figure 14. Public Office Hours Listing	26
Figure 15. Glassmorphic Login Screens	26
Figure 16. Glassmorphic Register Screens.....	27
Figure 17. Password Recovery Flow	27
Figure 18. Role-Aware Dashboard Home	28
Figure 19. My Profile & Security	28
Figure 20. Notification Center.....	29
Figure 21. My Bookings & Timeline View.....	29
Figure 22. Lecturer Search & Directory	30
Figure 23. Import AAO PDF Dashboard	30
Figure 24. Time Table View.....	31
Figure 25. Weekly Slot Picker.....	31
Figure 26. Group Booking & Recurring Setup.....	32
Figure 27. Waitlist Management Dashboard.....	32
Figure 28. Availability Rules Manager	33
Figure 29. Slot Waitlist Queue	33
Figure 30. My Teaching Schedule.....	34
Figure 31. Booking Review & Attendance Journal.....	34
Figure 32. User & Semester Management	35
Figure 33. PDF Timetable Importer	35

Figure 34. Allocation Policy Manager & Audit Log.....36

Figure 35. Analytics & Policy Benchmark Runner.....36

LIST OF TABLES

Table 1. User Table	19
Table 2. Role Table	20
Table 3. Semester Table	20
Table 4. Availability Rule Table	20
Table 5. Schedule Entry Table	20
Table 6. Slot Table	21
Table 7. Booking Table	21
Table 8. Booking Participants Table	22
Table 9. Waitlist Entry Table	22
Table 10. Allocation Event Table.....	22
Table 11. Notification Table.....	22

LIST OF ABBREVIATIONS

No.	Term	Meaning
1	ACID	Atomicity, Consistency, Isolation, Durability
2	AI	Artificial Intelligence
3	AMS	Asset Management Software
4	API	Application Programming Interface
5	CPU	Central Processing Unit
6	CSS	Cascading Style Sheets
7	CSV	Comma-Separated Values
8	DTO	Data Transfer Object
9	EIU	Eastern International University
10	FCFS	First-Come, First-Served
11	FIFO	First-In, First-Out
12	FK	Foreign Key
13	GB	Gigabyte
14	GiST / GIST	Generalized Search Tree
15	HD	High Definition
16	HTML	HyperText Markup Language
17	HTTP	Hypertext Transfer Protocol
18	HYBRID	Hybrid Allocation Model
19	IEEE	Institute of Electrical and Electronics Engineers
20	IoC	Inversion of Control
21	ISO	International Organization for Standardization
22	JPA	Java Persistence API
23	JSON / JSONB	JavaScript Object Notation / Binary JSON
24	JWT	JSON Web Token
25	LMS	Learning Management System
26	LTS	Long-Term Support
27	NEED	Priority-by-Need
28	ORM	Object-Relational Mapping
29	PDF	Portable Document Format
30	PK	Primary Key
31	QR	Quick Response
32	RAM	Random Access Memory
33	RBAC	Role-Based Access Control

34	REST / RESTful	Representational State Transfer
35	RSC	React Server Components
36	SEO	Search Engine Optimization
37	SIS	Student Information System
38	SMTP	Simple Mail Transfer Protocol
39	SQL	Structured Query Language
40	SSE	Server-Sent Events
41	SSD	Solid-State Drive
42	SSR	Server-Side Rendering
43	UI	User Interface
44	URI	Uniform Resource Identifier
45	UTC	Coordinated Universal Time

CHAPTER 1. OVERVIEW

1.1. Introduction

Effective coordination of academic consultation hours is essential for ensuring operational efficiency and educational equity within higher education institutions. Recognizing the persistent challenges faced by universities in managing advisor timetables, preventing schedule overlaps, and distributing consultation slots fairly, our team has developed OfficeHours. This solution is designed to provide a comprehensive and domain-aware platform that facilitates automated conflict detection, streamlined request processing, and equitable resource allocation.

The primary aim of this project is to deliver a robust software system that addresses key scheduling and allocation requirements while prioritizing usability, performance, and data integrity. By leveraging a modern technology stack – including Next.js and React for responsive web interface development, TypeScript for static typing reliability, and PostgreSQL with advanced exclusion constraints for data consistency – OfficeHours ensures a scalable, maintainable, and highly reliable architecture. This approach eliminates double-booking under heavy load and promotes long-term adaptability.

Throughout the project lifecycle, from initial requirement analysis and domain modeling to system implementation and empirical evaluation, our work has emphasized adherence to software engineering best practices and academic standards. The result is a dependable, user-centric solution capable of eliminating timetable conflicts, optimizing queuing workflows, and establishing quantifiable fairness through algorithmic scheduling policies.

This project not only addresses current operational friction in academic advising but also provides a strong foundation for future technological enhancements. By delivering a flexible and algorithmic scheduling platform, OfficeHours contributes to improving institutional productivity, enhancing student satisfaction, and establishing equitable access to academic resources.

1.2. Project Objectives

The objective of the OfficeHours project is to develop a comprehensive and user-centric platform that facilitates efficient management and fair allocation of academic consultation hours, whether for routine advising or high-demand course sessions. This system aims to provide a seamless and responsive interface that allows students and faculty to easily schedule slots, automatically resolve timetable conflicts, and track consultation requests, thereby enhancing transparency and control over academic advising resources. By leveraging advanced technologies such as Next.js and React for web interface development, TypeScript for static

typing, and PostgreSQL for reliable database management, the project ensures a scalable and maintainable architecture tailored to meet evolving institutional needs.

A key focus of the OfficeHours project is to deliver a solution that balances robust algorithmic performance with intuitive usability. The platform is designed to support personalized scheduling workflows, clear navigation, and real-time status updates, enabling users to access availability and waitlist information quickly and accurately. Additionally, considerable emphasis is placed on transactional correctness and data integrity, with database-level safeguards – such as PostgreSQL exclusion constraints – implemented to prevent double-booking anomalies, ensure concurrency control under peak loads, and protect institutional scheduling data. These features collectively contribute to building user trust and promoting effective academic consultation practices.

Moreover, the OfficeHours system is built with research versatility and future institutional growth in mind. Its modular architecture features a pluggable allocation engine that allows for the implementation and empirical evaluation of diverse fairness policies, including FCFS, Priority-by-Need, and Hybrid models. This flexibility ensures that the platform can measure equity metrics – such as the Gini coefficient and wait-time variance – and adapt to changing institutional requirements over time. Ultimately, the OfficeHours project aims to empower higher education institutions to optimize consultation resource distribution, reduce structural access disparities, and make informed administrative decisions through data-driven insights, positioning the software as a vital tool for sustainable academic success.

1.3. Challenges

During the development of the OfficeHours platform, our team encountered several technical and architectural challenges that required careful consideration and effective problem-solving. One major challenge was adapting to the integration of modern web technologies and asynchronous communication channels, such as Next.js for responsive frontend interfaces, SSE for real-time notifications, and client-side PDF timetable parsing, which demanded a cohesive architectural design to ensure seamless data flow between components. Another significant challenge involved identifying and resolving concurrency anomalies – specifically double-booking under peak consultation demand – requiring the implementation of advanced PostgreSQL GiST exclusion constraints to maintain database transactional correctness and deliver a smooth user experience. Additionally, designing and fine-tuning the pluggable allocation engine to compute live Gini equity metrics without direct institutional SIS integration required optimizing algorithmic performance and enforcing strict data security protocols.

Despite these obstacles, our team’s dedication and collaborative efforts enabled us to overcome technical difficulties and develop a scalable, reliable, and equitable scheduling solution.

1.4. Report Structure

This report is structured into several chapters, each addressing key components of the OfficeHours project. It begins with the Technological Stack chapter, which explains the reasoning behind the selection of tools and frameworks such as Next.js, React, TypeScript, and PostgreSQL. Following this, the Application Analysis chapter outlines the project requirements and provides an overview of the system’s data architecture, functional specifications, and database relational models. The Design and Implementation section details the overall system architecture, user interface logic, and core scheduling mechanisms. The Experiment and Discussion chapter presents the empirical evaluation of the slot-allocation policies, testing results, and system performance measured by the Gini coefficient. Finally, the report concludes with the Conclusion and Future Work chapter, summarizing the project’s achievements and suggesting areas for further enhancement and expansion.

CHAPTER 2. INTRODUCTION TO TECHNOLOGIES

2.1. Next.js and React

2.1.1. Introduction to Next.js and React

React and Next.js form the core front-end framework for the OfficeHours platform. Built on React 19 and Next.js 16 using the App Router architecture, route groups ((auth), (dashboard)) separate unauthenticated public routes from authenticated dashboard routes, with Server Components used for metadata and Suspense boundaries around client-rendered page content. Because the current build has no backend, the dashboard itself is fully client-rendered against local mock data; auth-gating is handled client-side in (dashboard)/layout.tsx via a loading skeleton rather than server-side middleware - a known limitation to close when the Spring Boot backend lands.

2.1.2. Advantages of Next.js and React

- Server-Side Rendering and Performance: Pre-renders pages and optimizes script execution to boost load times and responsiveness.
- App Router Architecture: Simplifies page-based routing and layout management through nested directory structures.
- Component Purity and State Management: Derives state during render phases to avoid cascade mutations and hydration errors.
- SEO and Accessibility: Delivers structured HTML directly from the server for search engine visibility and accessibility compliance.

2.2. Tailwind CSS

2.2.1. Introduction to Tailwind CSS

Tailwind CSS provides the utility-first styling system for the web client, governed by custom semantic design tokens. Instead of arbitrary color codes, the design system enforces a validated color ramp, mapping interactive elements to brand blue (*--brand-500*) and operational statuses to dedicated semantic hues (amber for pending, green for confirmed, red for cancelled, and violet for waitlisted).

2.2.2. Advantages of Tailwind CSS

- Tokenized Design Consistency: Enforces standardized color ramps, typography, and spacing across all components.
- High Data Density: Maintains a flat design shell that prioritizes administrative readability.
- Rapid UI Iteration: Eliminates custom CSS boilerplate by utilizing utility classes directly within components.

2.3. Framer Motion and UI Libraries

2.3.1. Introduction to Framer Motion and UI Libraries

The user experience is enhanced using Framer Motion alongside specialized visual libraries including Lucide Icons, Recharts, and Rough.js. Framer Motion manages layout transitions and entrance animations, while Recharts handles quantitative rendering for access equity metrics. Rough.js applies Neo-Brutalist canvas graphics on public landing pages to distinguish marketing content from internal administrative tools.

2.3.2. Advantages of UI Libraries

- Fluid Layout Transitions: Framer Motion provides smooth component animations and stagger effects.
- Accessibility Integration: Respects *useReducedMotion()* settings to disable heavy animations for sensitive users.
- Data Visualization Precision: Recharts renders single-hue analytical charts for Gini coefficient tracking without decorative clutter.
- Lightweight Vector Graphics: Lucide provides tree-shakeable icons that dynamically adapt to active design tokens.

2.4. PDF.js

2.4.1. Introduction to PDF.js

PDF.js (pdfjs-dist) handles client-side parsing of university timetables within the web client. By running inside a non-blocking Web Worker, it parses class schedules directly on the student's browser using spatial clustering algorithms, mapping raw text blocks into structured JSON time blocks without offloading heavy computing tasks to the backend.

2.4.2. Advantages of PDF.js

- Client-Side Processing: Eliminates server burden by extracting PDF schedule data locally.
- Privacy-Preserving: Processes sensitive student timetable documents inside the client browser context.
- Deterministic Parsing: Converts spatial text coordinates into structured timetable representations accurately.

2.5. Java Spring Boot

2.5.1. Introduction to Java Spring Boot

The backend infrastructure is constructed as a high-performance, modular monolith utilizing Java and the Spring Boot framework. Designed as a RESTful Web API, Spring Boot centralizes schedule management, conflict detection, and algorithmic slot allocation. The

system leverages Spring Data JPA for robust relational data management and Jakarta Validation middleware for request payload verification. All chronological data exchange is strictly standardized using ISO-8601 UTC formats to guarantee multi-timezone consistency.

2.5.2. Advantages of Java Spring Boot

- Enterprise Architecture: Provides a robust modular design with IoC and dependency injection for long-term maintainability.
- Strong Security Framework: Integrates natively with Spring Security to enforce fine-grained authentication and RBAC.
- ORM and Transaction Management: Simplifies database operations via Spring Data JPA / Hibernate while guaranteeing transactional integrity.
- High Performance and Scalability: Efficiently handles high-concurrency requests and complex business logic execution.

2.6. PostgreSQL

2.6.1. Introduction to PostgreSQL

PostgreSQL 15+ serves as the primary relational database, providing ACID-compliant data storage and advanced constraint enforcement. Utilizing extensions such as *btree_gist* and *citext*, PostgreSQL enforces database-level exclusion constraints that mathematically prevent double-booking and maintain case-insensitive email uniqueness.

2.6.2. Advantages of PostgreSQL

- ACID Integrity: Guarantees absolute transactional correctness during concurrent scheduling requests.
- GiST Exclusion Constraints: Combines scalar logic with time range overlaps (*tstzrange*) to block double-booking at the storage layer.
- Timezone Precision: Native *timestampz* data types protect chronological accuracy across client regions.

2.7. JSON Web Token

2.7.1. Introduction to JSON Web Token

JSON Web Token is deployed for stateless user authentication and RBAC. Passwords are irreversibly hashed using *bcrypt* before storage. Issued tokens carry cryptographically signed identity payloads that govern access across STUDENT, LECTURER, and ADMIN roles alongside row-level data sovereignty checks.

2.7.2. Advantages of JSON Web Token

- Stateless Authentication: Eliminates server-side session lookup overhead for API requests.

- Fine-Grained RBAC: Enforces endpoint authorization and user-specific row-level security.
- Cryptographic Security: Prevents token tampering and account enumeration through signed signatures and uniform response codes.

2.8. Git and GitHub

2.8.1. Introduction to Git and GitHub

Git and GitHub provide source code management, version tracking, and collaborative development tools. Database structural changes are managed via sequential migration scripts tracked within GitHub, keeping local, staging, and production environments synchronized.

2.8.2. Advantages of GitHub

- Version Control and Collaboration: Tracks historical code changes and supports multi-developer workflows.
- Relational Schema Migration: Ensures schema parity across deployment environments via versioned database migrations.

2.9. System Architecture and Technology Stack

2.9.1. Front-end

- Next.js & React: Core web framework handling server-side rendering, routing, and component architecture.
- Tailwind CSS: Utility-first styling framework with custom semantic token configuration.
- Framer Motion: Layout animation library with integrated accessibility motion controls.
- Recharts & Lucide: Data visualization engine for equity metrics and vector icon set.
- PDF.js: Web Worker-based client parser for PDF timetable ingestion.

2.9.2. Back-end

- Java Spring Boot: Modular RESTful backend API handling scheduling logic, allocation policies, and transaction management.
- Spring Security & JWT: Stateless authentication framework providing role-based security and password encryption via BCrypt.
- PostgreSQL 15+: Relational database with *btree_gist* exclusion constraints for conflict prevention.
- GitHub & Migrations: Version control repository and database migration tools.

CHAPTER 3. APPLICATION ANALYSIS, DESIGN AND IMPLEMENTATION

3.1. System Description

The OfficeHours Academic Consultation Platform is a specialized system engineered to automate, balance, and optimize consultation scheduling between students and faculty members. Built on a high-performance Java Spring Boot RESTful Web API backend, the application integrates a modern Next.js client interface utilizing Tailwind CSS to deliver an accessible and responsive user experience. PostgreSQL serves as the underlying relational database, utilizing exclusion constraints (*btree_gist*) to mathematically guarantee zero double-booking at the storage level, complemented by Redis for caching computationally demanding availability calculations.

The platform is structured into distinct, role-tailored interfaces designed around the specific responsibilities of Students, Lecturers, and Administrators:

- Student Interface: Tailored for seamless scheduling and interaction tracking. Students can browse lecturer availability directories, upload PDF class timetables to automatically filter out scheduling conflicts, request individual or group consultation slots, and join automated waitlists for oversubscribed sessions. The personal dashboard provides real-time status updates on submitted requests, upcoming appointments, and waitlist allocations.
- Lecturer Interface: Focused on supply-side availability management and appointment control. Lecturers can establish recurring weekly availability templates, overlay single-day exceptions (blocking or adding slots), review incoming appointment requests, mark student attendance post-session, and maintain meeting logs.
- Administrator Interface: Designed for global system governance and algorithmic allocation oversight. Administrators manage user accounts and academic semester cycles, execute bulk timetable imports via PDF parsing, configure dynamic fairness policies for waitlist allocation, and access analytical dashboards monitoring resource distribution equity across the student body via Gini coefficient metrics.

By combining real-time communication, role-driven workflows, and algorithmic fairness engines, the OfficeHours platform streamlines academic advisement, eliminates scheduling friction, and ensures transparent, equitable access to faculty mentorship.

3.2. Use Case Diagram

3.2.1. General Use Case Diagram

The OfficeHours platform supports core operational functions divided across system roles to streamline academic advising workflows.

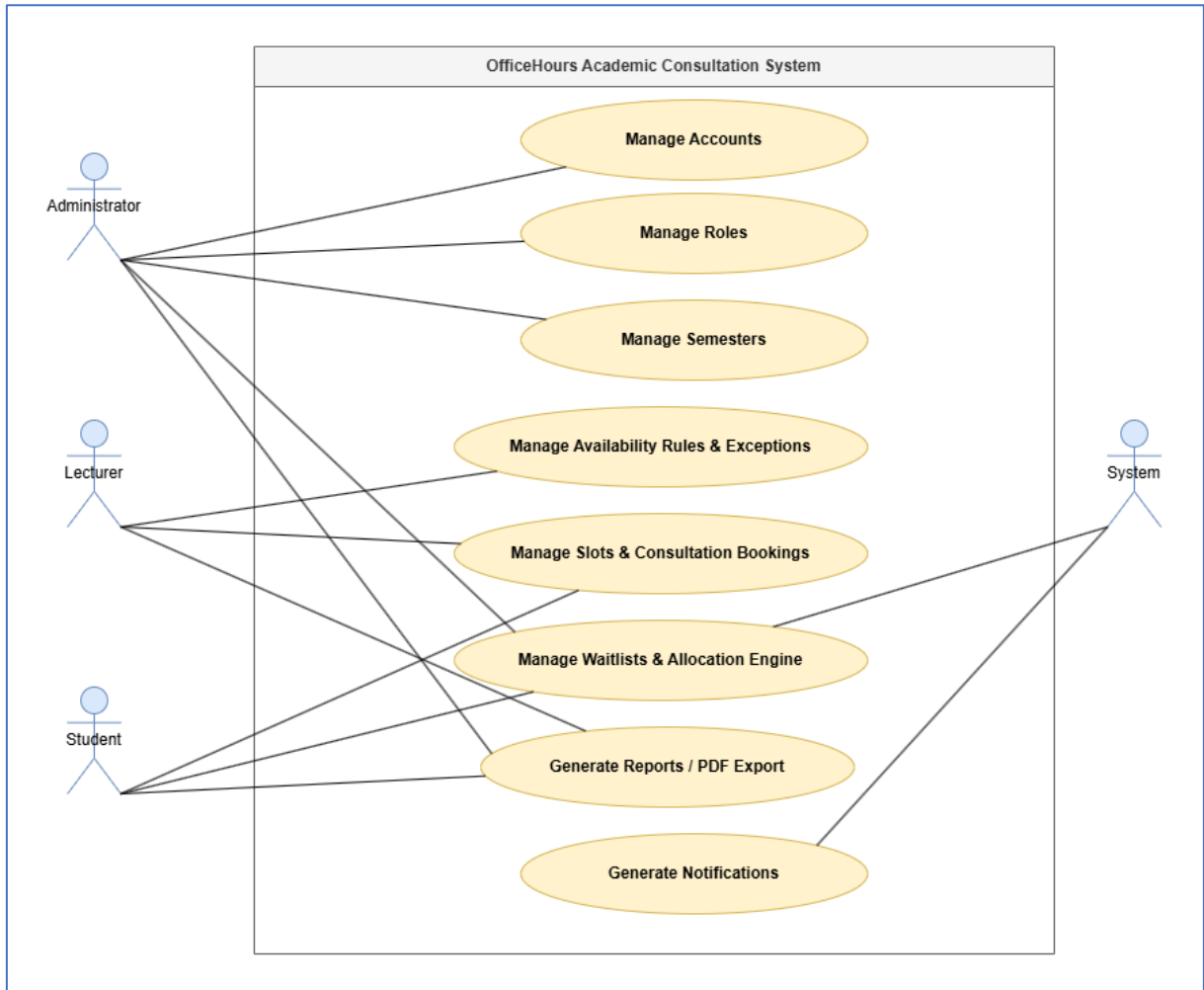


Figure 1. OfficeHours Academic Consultation System Diagram

- **Manage Accounts:** Allows the Administrator to create, update, deactivate, and assign roles (STUDENT, LECTURER, ADMIN) to system users, ensuring secure, role-restricted operational access.
- **Manage Roles:** Enables Administrators to configure permission boundaries and access controls for different user tiers.
- **Manage Semesters:** Allows Administrators to define, activate, and archive academic term boundaries that govern recurring scheduling patterns.
- **Manage Availability Rules & Exceptions:** Enables Lecturers to establish weekly recurring availability hours and overlay specific date exceptions (adding extra slots or blocking unavailable periods).

- **Manage Slots & Consultation Bookings:** Available to both Students and Lecturers. Students browse conflict-filtered slots and submit booking requests, while Lecturers review, confirm, or decline pending requests.
- **Manage Waitlists & Allocation Engine:** Allows Students to join waitlists for full slots, while the System and Administrator manage automated algorithmic allocations and manual overrides.
- **Generate Reports / PDF Export:** Enables Users and Administrators to export consultation histories, timetable entry parses, and allocation audit logs in PDF or CSV format for documentation and academic research.
- **Generate Notifications:** Automated alerts generated by the System via SSE and email (SMTP) to notify users of booking confirmations, cancellations, waitlist offer dispatches, and pending requests.

3.2.2. For Administrator

The Administrator controls system operations, user authorization, and global configuration parameters.

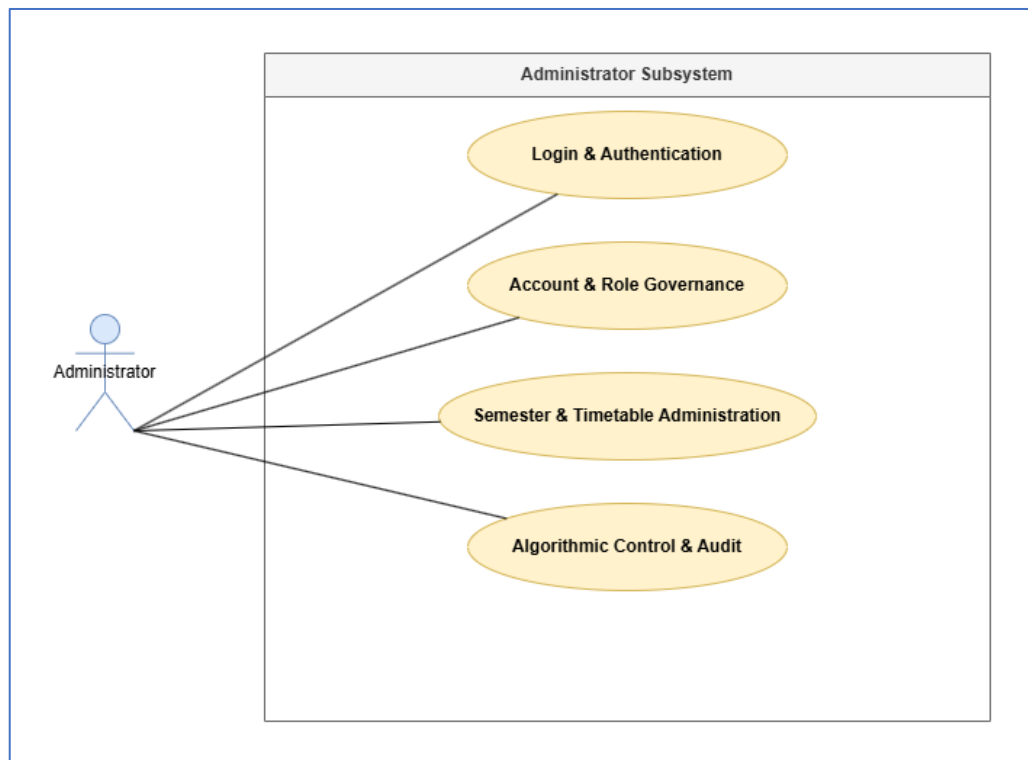


Figure 2. Administrator Subsystem Diagram

- **Login & Authentication:** Supported by credentials validation and password recovery workflows.

- **Account & Role Governance:** Admin views full account registries, creates new user profiles, updates organizational roles/departments, and activates or deactivates user access.
- **Semester & Timetable Administration:** Sets active semester boundaries and runs the same in-browser PDF timetable scanner used by Students and Lecturers, on behalf of any user who can't self-serve - a Lecturer/Student role toggle selects the account, then either an AAO PDF upload or a manual entry form writes directly into that person's schedule; a separate Slot Search tool cross-checks open slots across all lecturers.
- **Algorithmic Control & Audit:** Selects active waitlist allocation policies, executes manual overrides when human intervention is required, and monitors Gini coefficient equity dashboards.

3.2.3. For Lecturer

Lecturers manage consultation availability, review appointment requests, and track advising outcomes.

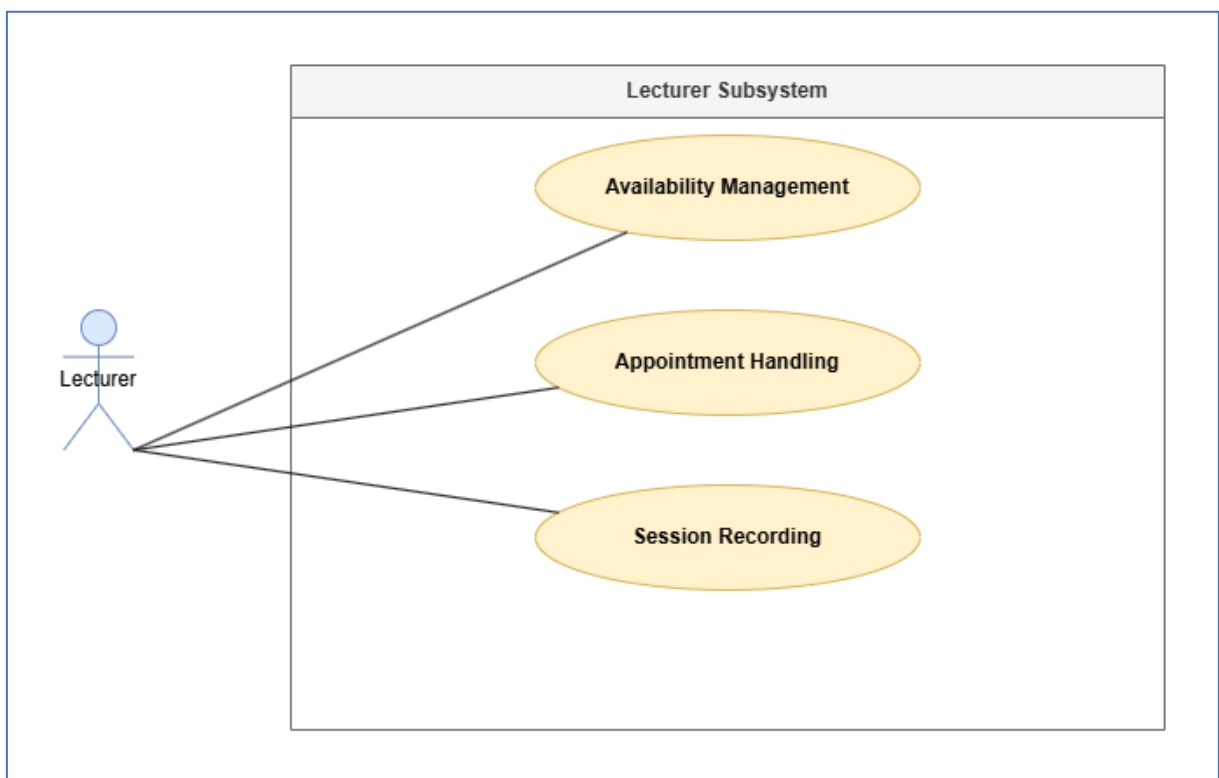


Figure 3. Lecturer Subsystem Diagram

Availability Management: Lecturers define recurring weekly consultation templates and add single-day schedule adjustments.

- **Schedule Synchronization:** Lecturers scan their own official AAO teaching timetable PDF in-browser (the same scanner Students use) to auto-populate their teaching schedule, or add and remove individual class blocks manually.

- Appointment Handling: Lecturers receive notifications of student booking requests and hold primary authority to accept or reject appointments.
- Session Recording: Lecturers mark attendance statuses (Attended, Absent, No-Show) and attach non-academic mentoring notes post-consultation.

3.2.4. For Student

Students browse availability, submit consultation requests, and track scheduling progress.

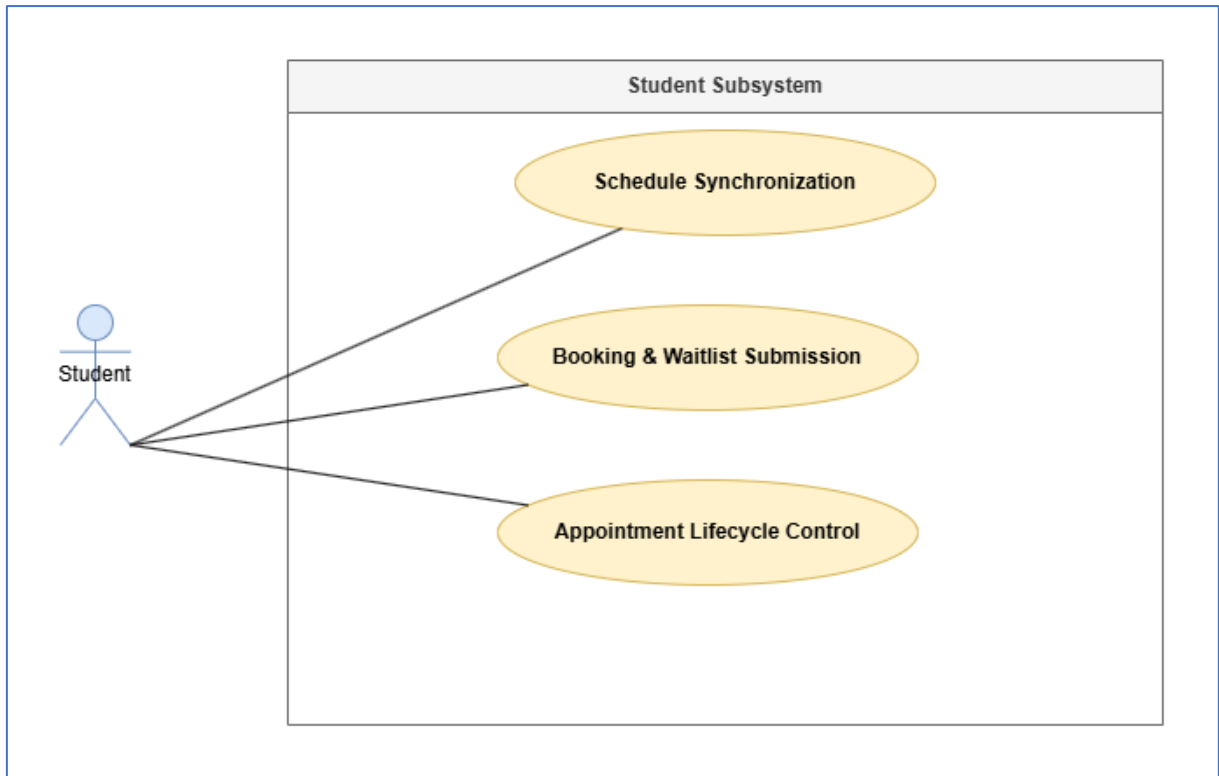


Figure 4. Student Subsystem Diagram

Schedule Synchronization: Students scan their official AAO class timetable PDF entirely in-browser (pdf.js, no server upload) to auto-populate a dual Timetable Matrix / Agenda List view of their own schedule, which dynamically masks unavailable times during slot searching; individual class blocks can also be added or removed manually.

- **Booking & Waitlist Submission:** Students request available consultation times, initiate group appointments with peers, or queue in automated waitlists when capacity is reached.
- **Appointment Lifecycle Control:** Students review request statuses and can cancel or reschedule appointments within pre-configured notice windows.

3.3. Activity Diagram

3.3.1. Login Function

Illustrates user authentication via the Spring Security and JWT framework.

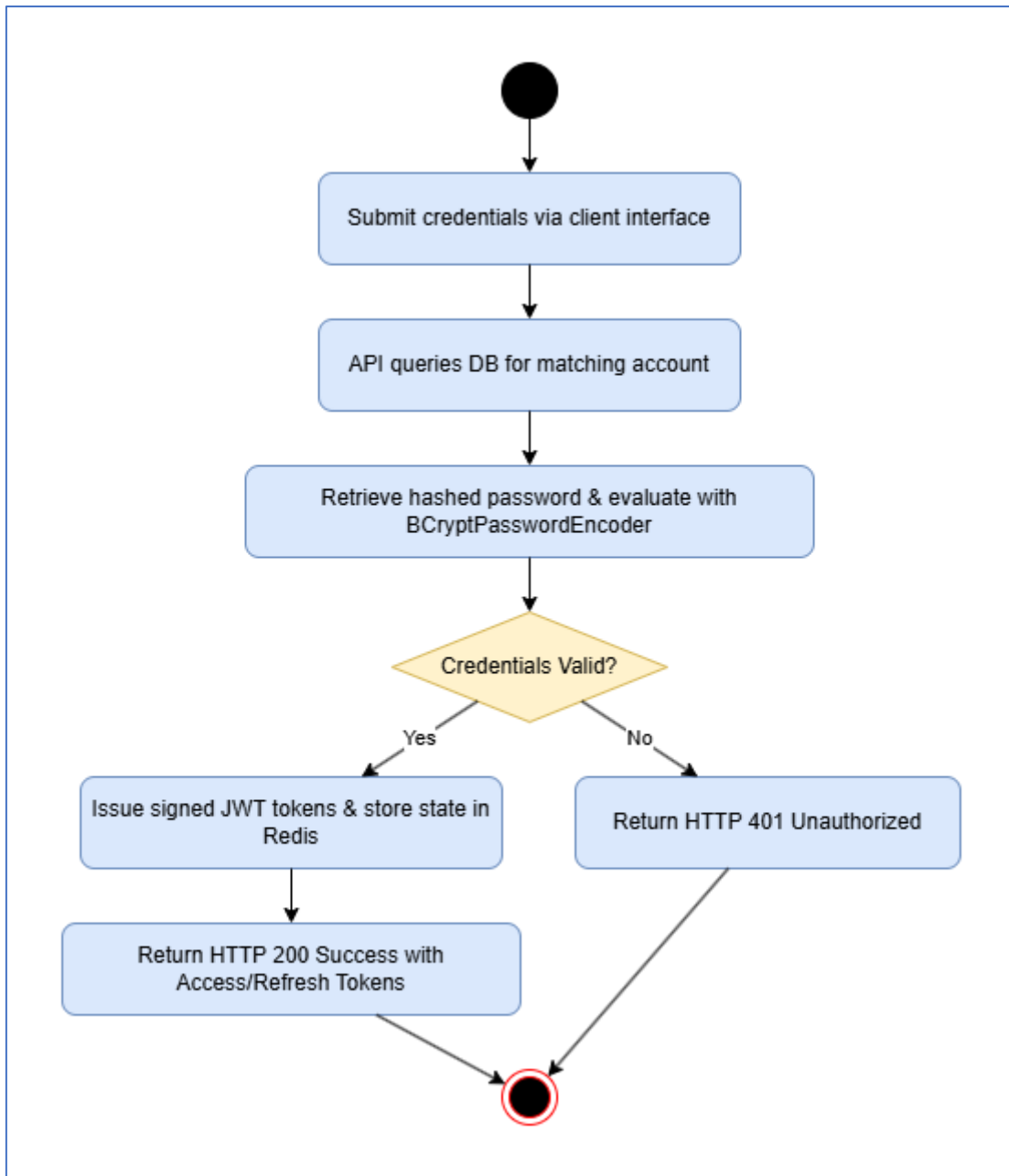


Figure 5. Login Function Diagram

The user submits credentials via the client login interface. The API queries the database for the matching record, retrieves the hashed password, and evaluates it using BCryptPasswordEncoder. Upon validation, the server issues signed JWT access and refresh tokens, storing token state in Redis. Invalid attempts trigger immediate HTTP 401 error responses.

3.3.2. Create Function

Outlines the generation of new entity records (e.g., availability slots or booking requests) with database-level concurrency validation.

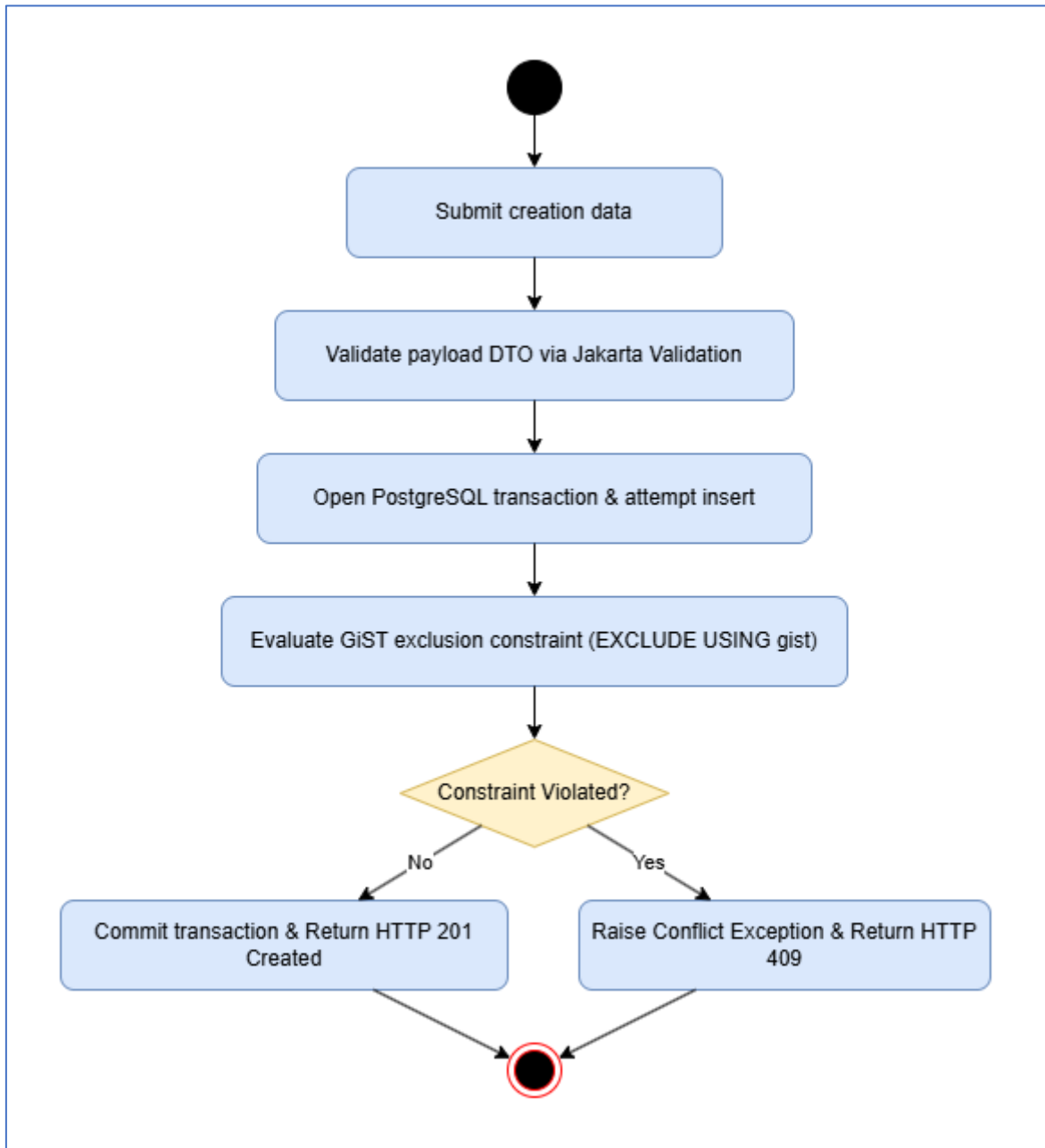


Figure 6. Create Function Diagram

When an authorized user submits creation data, the Spring Boot API validates the payload DTO using Jakarta Validation. The backend opens a transaction to insert the record into PostgreSQL. If creating a booking, PostgreSQL evaluates the GiST exclusion constraint (EXCLUDE USING gist). Valid inserts commit and return HTTP 201 Created; overlapping entries raise a conflict exception, returning HTTP 409

3.3.3. Read Function

Illustrates fetching and filtering slot or user data utilizing Redis caching.

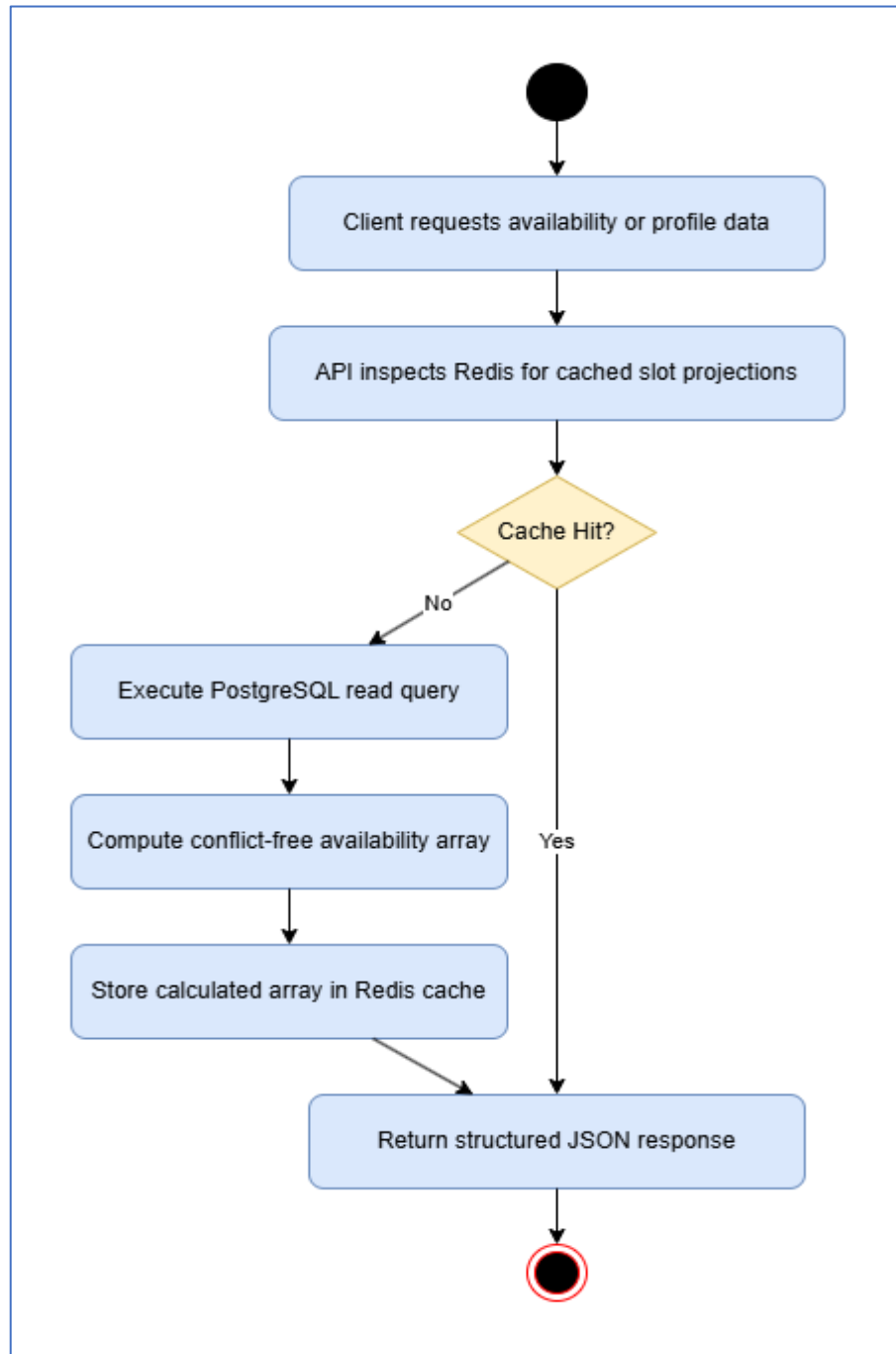


Figure 7. Read Function Diagram

The client requests availability or profile data. The API inspects Redis for cached slot projections. On a cache hit, data returns immediately. On a cache miss, the system executes a PostgreSQL read query, computes conflict-free availability by subtracting schedule blocks, caches the calculated array in Redis, and returns structured JSON.

3.3.4. Update Function

Outlines modification of existing entities, such as adjusting availability exceptions or updating booking statuses.

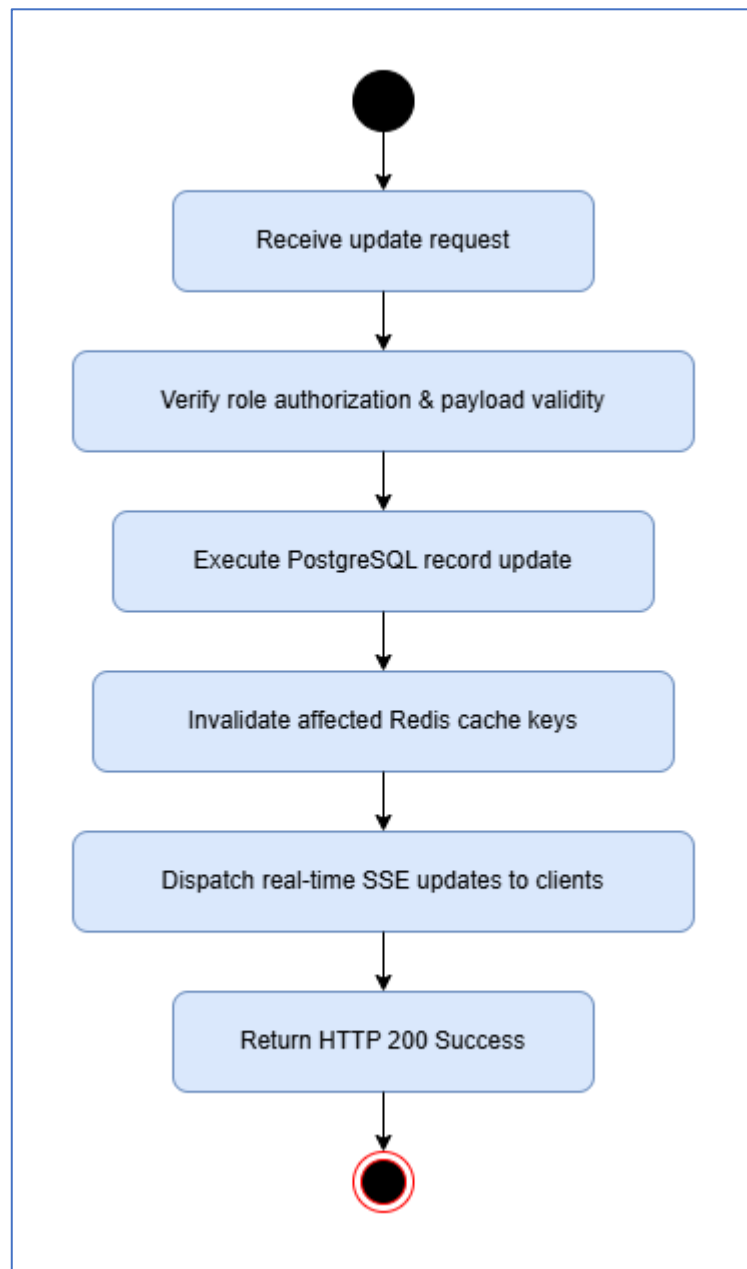


Figure 8. Update Function Diagram

Upon receiving an update request, the system verifies role authorization and checks payload validity. Once PostgreSQL updates the database record, Spring Boot invalidates affected Redis cache keys and dispatches real-time updates through SSE to connected clients.

3.3.5. Delete Function

Describes entity removal or booking cancellation governed by business rules.

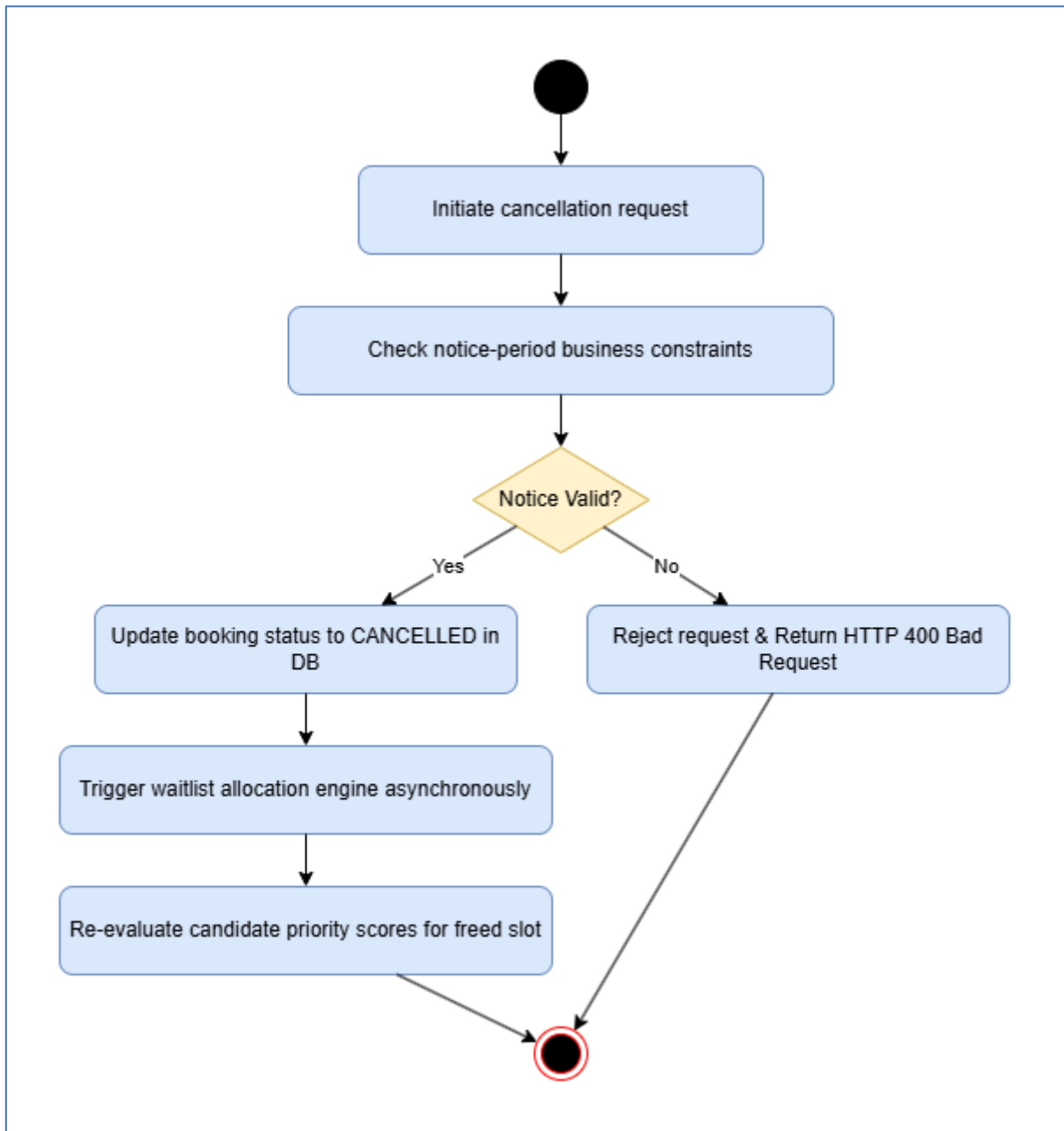


Figure 9. Delete Function Diagram

When a user initiates a cancellation, the backend checks whether the request satisfies configurable notice-period constraints. If valid, the system updates the booking status to CANCELLED. This status update asynchronously triggers the waitlist allocation engine to re-evaluate candidate priority scores for the newly freed slot.

3.3.6. Password Recovery

Illustrates defensive, token-based account password recovery.

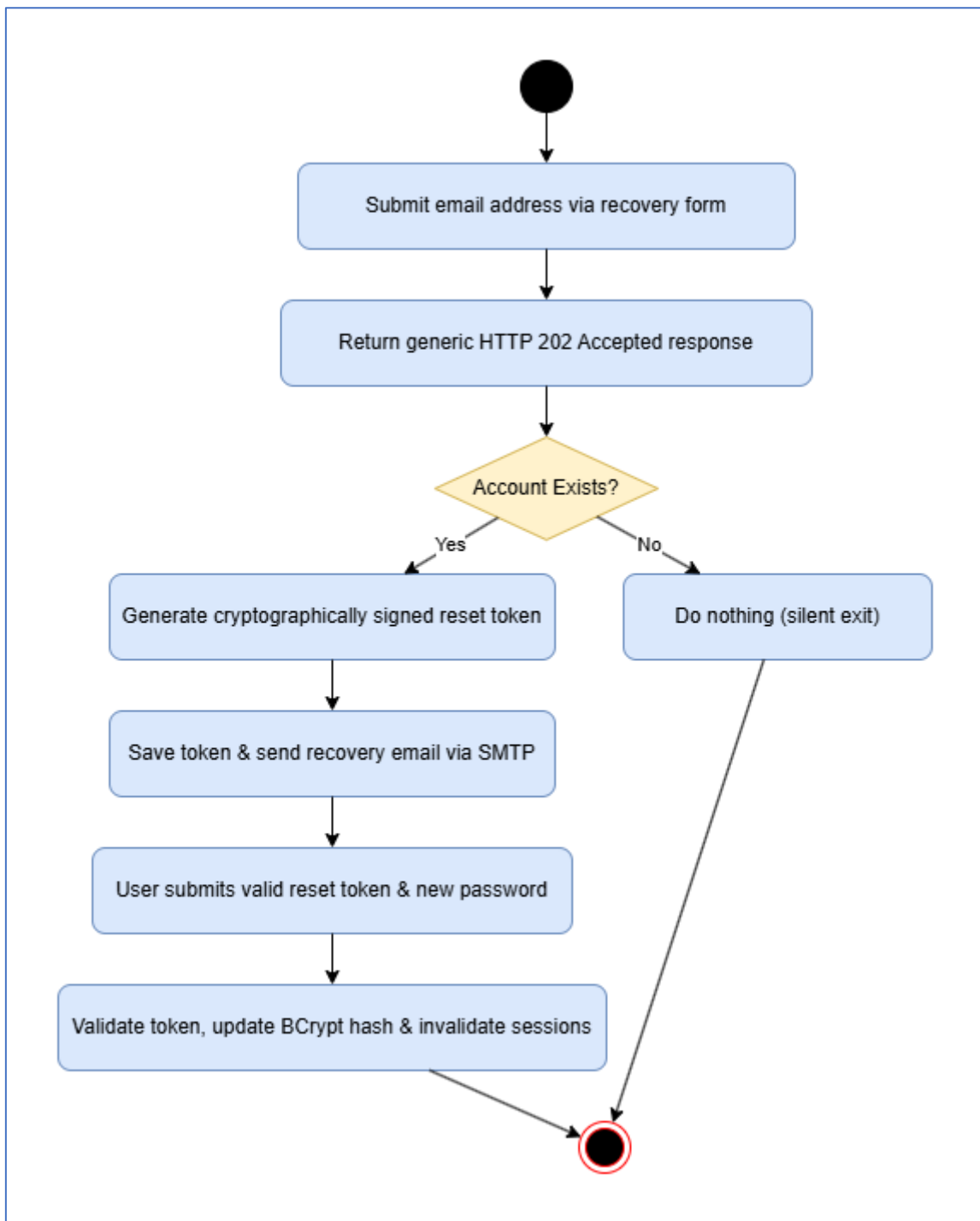


Figure 10. Password Diagram

The user submits an email address. The system returns a generic HTTP 202 Accepted response regardless of whether the account exists to thwart account enumeration attacks. If the account exists, a cryptographically signed reset token is saved and sent to the user's inbox via SMTP. Submitting a valid token allows setting a new password, updating the BCrypt hash, and invalidating existing user sessions.

3.4. Database

3.4.1. Entity-Relationship Diagram

The relational database design centers around core domain entities governing identities, schedules, consultation slots, bookings, waitlists, and audit logs.

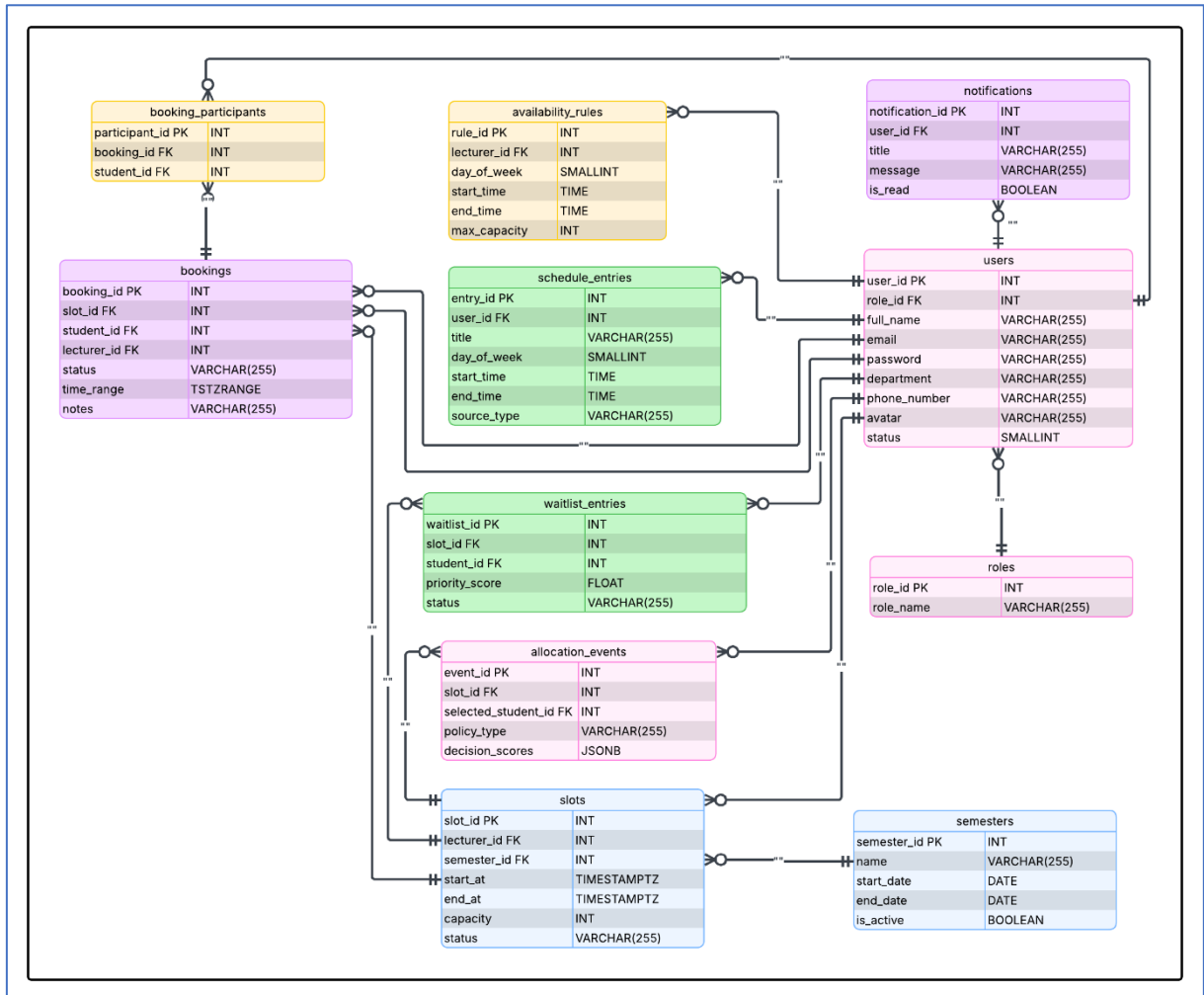


Figure 11. Entity-Relationship Diagram

3.4.2. Database Analysis

User Table: Stores profile credentials and role assignments.

Table 1. User Table

No.	Field Name	Type	Description
1	user_id	int (PK)	Unique primary key identifier
2	role_id	int (FK)	References roles(role_id)
3	full_name	varchar(255)	User full name
4	email	varchar(255)	Unique case-insensitive email address
5	password	varchar(255)	BCrypt encrypted password hash
6	department	varchar(255)	Academic department affiliation

7	phone_number	varchar(255)	Contact telephone number
8	avatar	varchar(255)	Profile image URI
9	status	tinyint	Account status (1 = Active, 0 = Inactive)

Role Table: Stores system access roles.

Table 2. Role Table

No.	Field Name	Type	Description
1	role_id	int (PK)	Unique role identifier
2	role_name	varchar(255)	Name of role (STUDENT, LECTURER, ADMIN)

Semester Table: Tracks active and historical academic terms.

Table 3. Semester Table

No.	Field Name	Type	Description
1	semester_id	int (PK)	Primary key identifier
2	name	varchar(255)	Semester title (e.g., "Fall 2026")
3	start_date	date	Term start boundary
4	end_date	date	Term end boundary
5	is_active	boolean	Flag for the currently active global semester

Availability Rule Table: Stores recurring weekly consultation templates defined by lecturers.

Table 4. Availability Rule Table

No.	Field Name	Type	Description
1	rule_id	int (PK)	Unique rule identifier
2	lecturer_id	int (FK)	Foreign key referencing users(user_id)
3	day_of_week	tinyint	Day index (1 = Monday, ..., 7 = Sunday)
4	start_time	time	Daily window start time
5	end_time	time	Daily window end time
6	max_capacity	int	Maximum allowed students per generated slot

Schedule Entry Table: Stores imported timetable busy blocks for students and lecturers.

Table 5. Schedule Entry Table

No.	Field Name	Type	Description
1	entry_id	int (PK)	Primary key identifier

2	user_id	int (FK)	Owner user identifier
3	title	varchar(255)	Course or event title
4	day_of_week	tinyint	Day index
5	start_time	time	Event start time
6	end_time	time	Event end time
7	source_type	varchar(255)	Import origin (PDF_PARSED, CSV_IMPORTED, MANUAL)

Slot Table: Represents materialized bookable consultation sessions.

Table 6. Slot Table

No.	Field Name	Type	Description
1	slot_id	int (PK)	Unique slot identifier
2	lecturer_id	int (FK)	Host lecturer ID
3	semester_id	int (FK)	Related academic semester ID
4	start_at	timestampz	Slot start timestamp in UTC
5	end_at	timestampz	Slot end timestamp in UTC
6	capacity	int	Student capacity limit
7	status	varchar(255)	Current slot state (OPEN, FULL, CLOSED)

Booking Table: Records appointment instances between students and consultation slots.

Table 7. Booking Table

No.	Field Name	Type	Description
1	booking_id	int (PK)	Primary key identifier
2	slot_id	int (FK)	Foreign key to slots(slot_id)
3	student_id	int (FK)	Student user ID
4	lecturer_id	int (FK)	Denormalized lecturer ID for exclusion constraints
5	status	varchar(255)	Booking status (PENDING, CONFIRMED, DECLINED, CANCELLED)
6	time_range	tstzrange	Denormalized PostgreSQL range for GiST index
7	notes	varchar(255)	Post-session notes or student request notes

Booking Participants Table: Supports multi-student group bookings for a single consultation appointment.

Table 8. Booking Participants Table

No.	Field Name	Type	Description
1	participant_id	int (PK)	Unique record identifier
2	booking_id	int (FK)	Primary booking reference
3	student_id	int (FK)	Additional participating student ID

Waitlist Entry Table: Tracks queued students waiting for freed consultation slots.

Table 9. Waitlist Entry Table

No.	Field Name	Type	Description
1	waitlist_id	int (PK)	Primary key identifier
2	slot_id	int (FK)	Target full slot ID
3	student_id	int (FK)	Queued student ID
4	priority_score	float	Computed priority metric for fair allocation
5	status	varchar(255)	Entry status (QUEUED, OFFERED, EXPIRED, ACCEPTED)

Allocation Event Table: Maintains an audit log of automated and manual waitlist allocation decisions.

Table 10. Allocation Event Table

No.	Field Name	Type	Description
1	event_id	int (PK)	Primary key identifier
2	slot_id	int (FK)	Subject slot ID
3	selected_student_id	int (FK)	Selected recipient student ID
4	policy_type	varchar(255)	Applied policy (GINI_FAIRNESS, FIFO, MANUAL_OVERRIDE)
5	decision_scores	jsonb	Serialized candidate priority scores for research audit

Notification Table: Stores system alerts and status updates dispatched to users.

Table 11. Notification Table

No.	Field Name	Type	Description
1	notification_id	int (PK)	Primary key identifier
2	user_id	int (FK)	Recipient user ID
3	title	varchar(255)	Brief alert headline

4	message	varchar(255)	Main notification payload
5	is_read	boolean	Read/Unread flag

3.5. Implementation

3.5.1. Implementation of Scrum

To maintain structured progress and team alignment throughout software development, the team adopted the Scrum agile project management framework. Work items were organized into fixed 2-week iterations (Sprints).

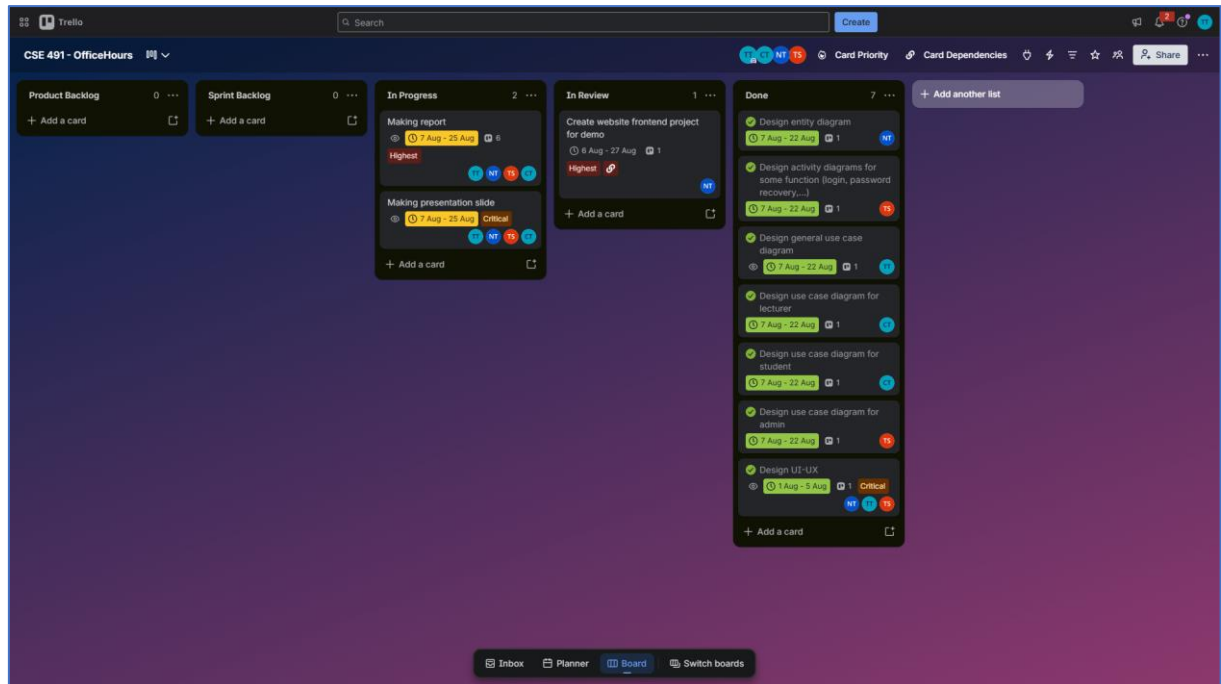


Figure 12. Trello – Project Management

- Sprint Planning: Prioritized backlog stories (e.g., implementing GiST exclusion constraints, configuring JWT stateless filters, building Next.js dashboards) into concrete Sprint Backlogs.
- Daily Stand-ups: Brief sync meetings conducted to track task completion, identify technical blockers, and coordinate API integration between front-end and back-end team members.
- Trello / Board Management: Visual Kanban boards tracked task progress across Product Backlog, Sprint Backlog, In Progress, Review, and Done statuses.
- Sprint Reviews & Retrospectives: End-of-sprint demonstrations evaluated completed increments against functional requirements, while retrospectives allowed the team to refine development workflows for subsequent iterations.

CHAPTER 4. EXPERIMENT AND DISCUSSION

4.1. Installation Environment

4.1.1. Hardware Requirements

To support the client-side development, rendering performance testing, and user interface evaluation of the Web Application, the development environment utilizes hardware that meets or exceeds the following specifications:

- Processor: Quad-core CPU (Intel Core i5 10th Gen, AMD Ryzen 5, or Apple M1/M2) to handle continuous client-side hot-reloading and web bundling workflows.
- Memory (RAM): Minimum of 8 GB (16 GB recommended) to maintain responsive performance while running browser developer tool suites, frontend development servers, and multiple client browser sessions.
- Storage: At least 10 GB of available SSD for node module dependencies, web asset packages, build outputs, and development tools.
- Display: 1080p Full HD resolution (1920x1080) or higher to verify responsive layouts across standard desktop and laptop screen dimensions.
- Network: Stable internet connection for downloading npm dependencies, CSS frameworks, and icon asset libraries.

4.1.2. Software Requirements

The software environment for building and running the frontend web interface includes:

- Operating System: Windows 10/11, macOS (Monterey 12.x or later), or Linux (Ubuntu 22.04 LTS).
- Runtime & Package Management: Node.js (v18.x or v20.x LTS) with npm or pnpm for executing client scripts and managing JavaScript/TypeScript packages.
- Development Environment: Visual Studio Code equipped with ESLint, Prettier, and Tailwind CSS extensions.
- Web Browsers: Google Chrome, Mozilla Firefox, and Microsoft Edge (latest stable versions) for cross-browser visual rendering and responsive testing.
- Frontend Libraries & Tooling:
 - + Next.js framework for modular UI component development.
 - + Tailwind CSS for utility-first responsive styling.
 - + Framer Motion for UI state transitions.
 - + Recharts for analytics visualization.
 - + pdf.js for client-side timetable parsing.

4.2. Results and Discussion

4.2.1. Web Interface Implementation

Public & Authentication Views:

- Landing Page: Implements a distinct "*Neo-Brutalist*" visual style featuring high-contrast borders, offset dark card shadows, and mathematical vector accent underlines rendered via rough.js.

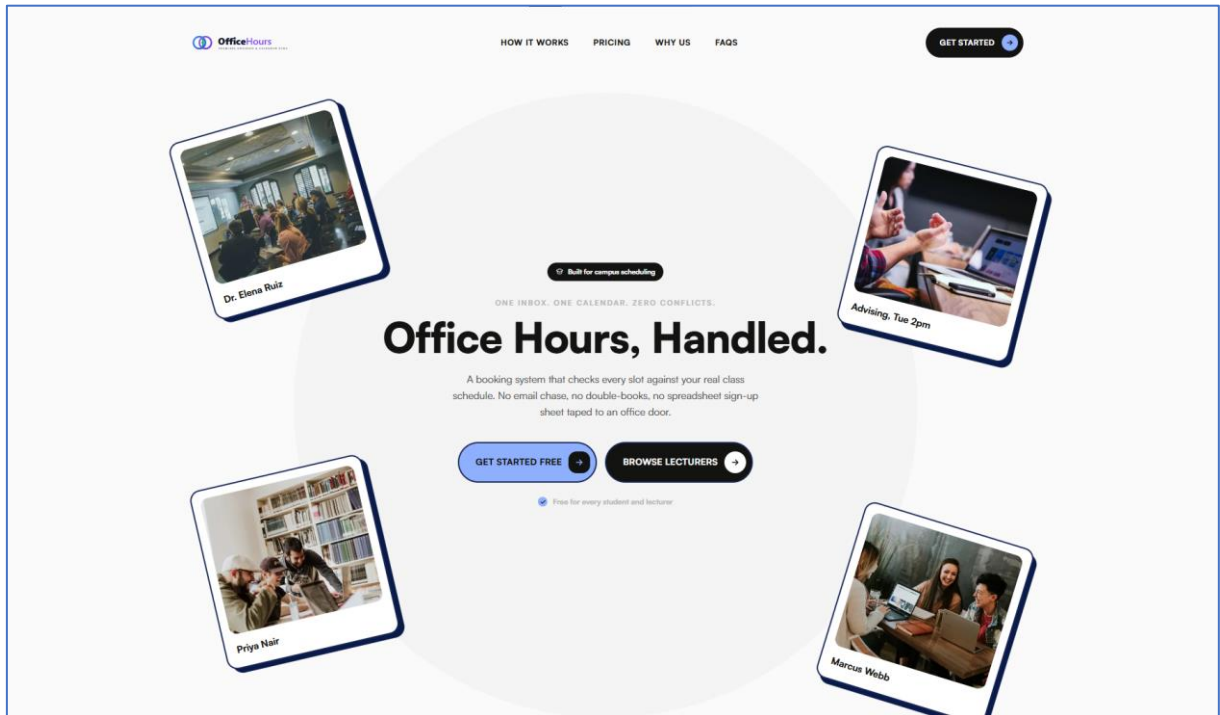


Figure 13. Landing Page

- Public Office Hours Listing: Renders a public view displaying open consultation slots across academic departments using a client-side mock data aggregation pipeline without requiring prior authentication.

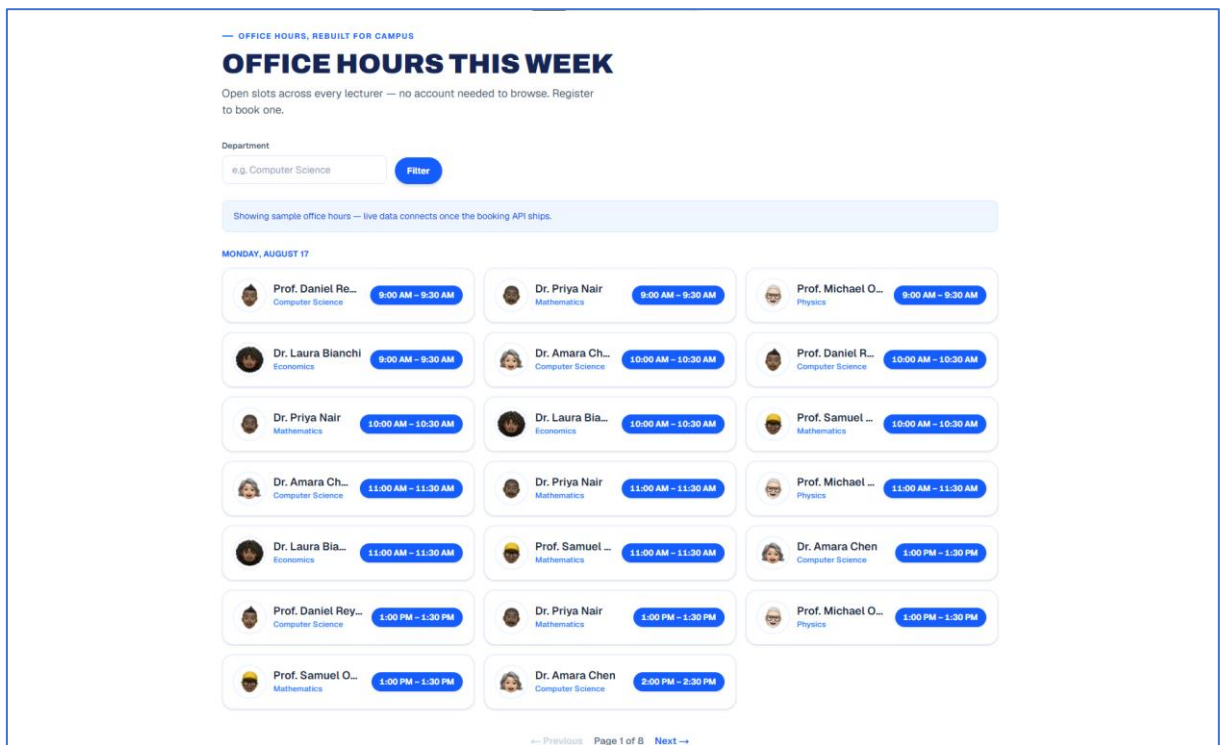


Figure 14. Public Office Hours Listing

- Glassmorphic Login & Register Screens: Utilizes CSS backdrop-filter: *blur(20px)* and semi-transparent containers over gradient backgrounds, incorporating client-side input validation for email formats and password criteria.

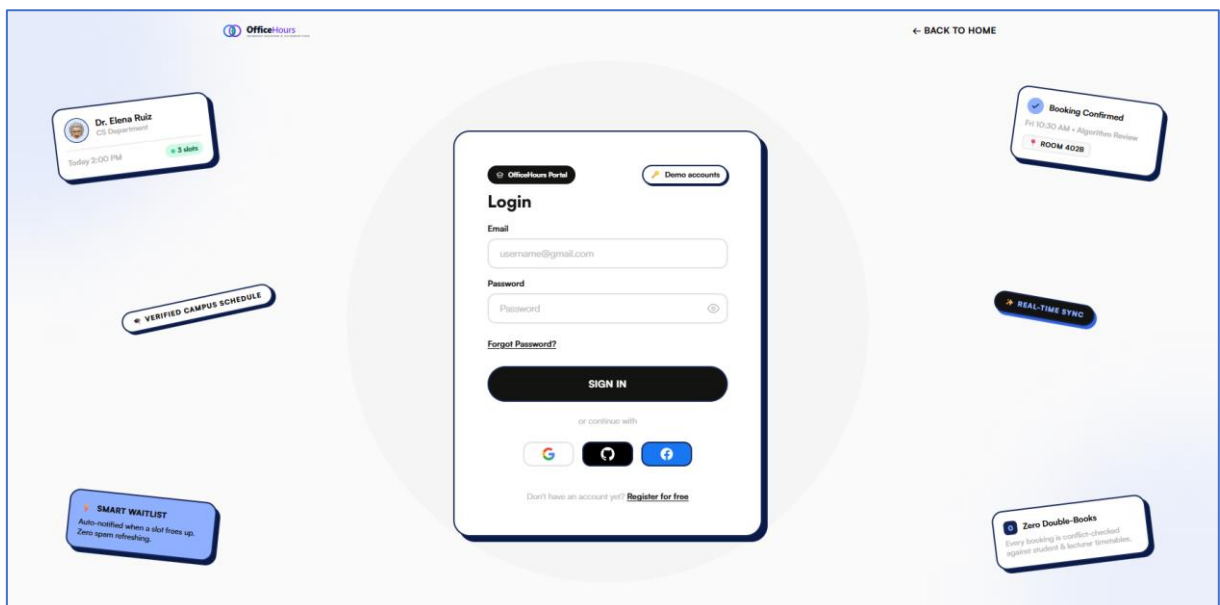


Figure 15. Glassmorphic Login Screens

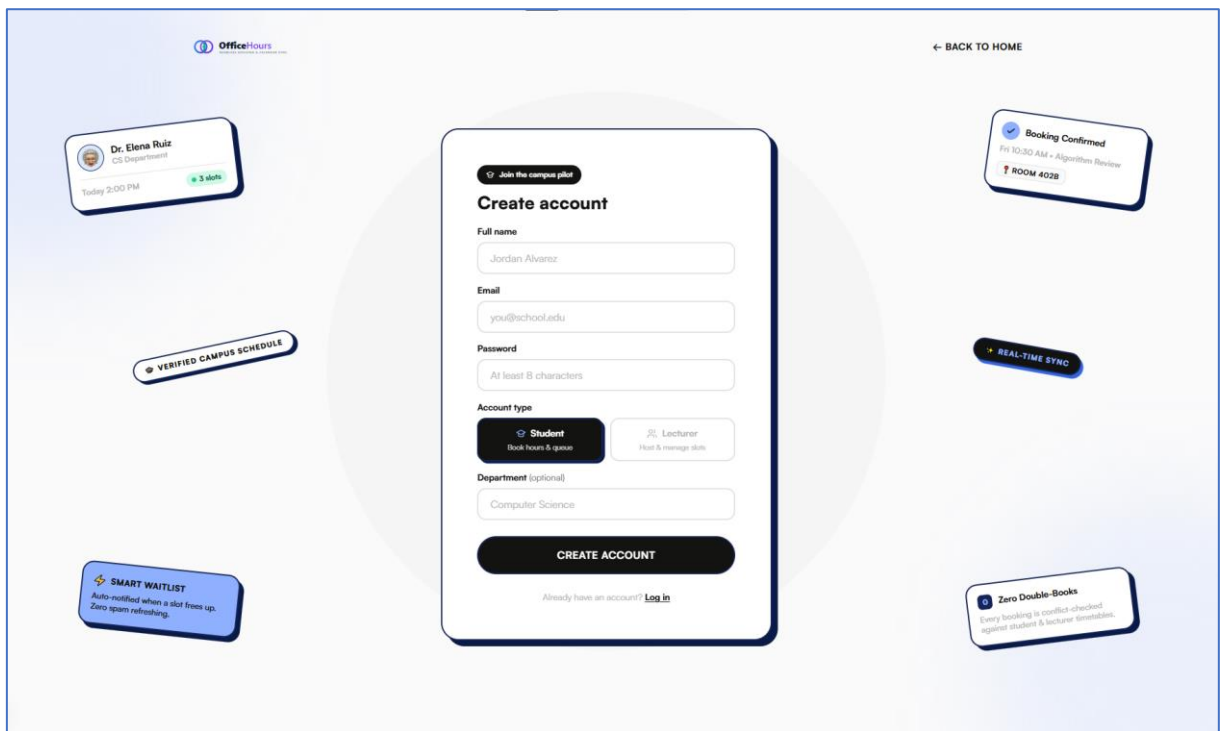


Figure 16. Glassmorphic Register Screens

- Password Recovery Flow: Provides interactive forms for password reset requests, incorporating immediate user feedback during email entry and token verification.

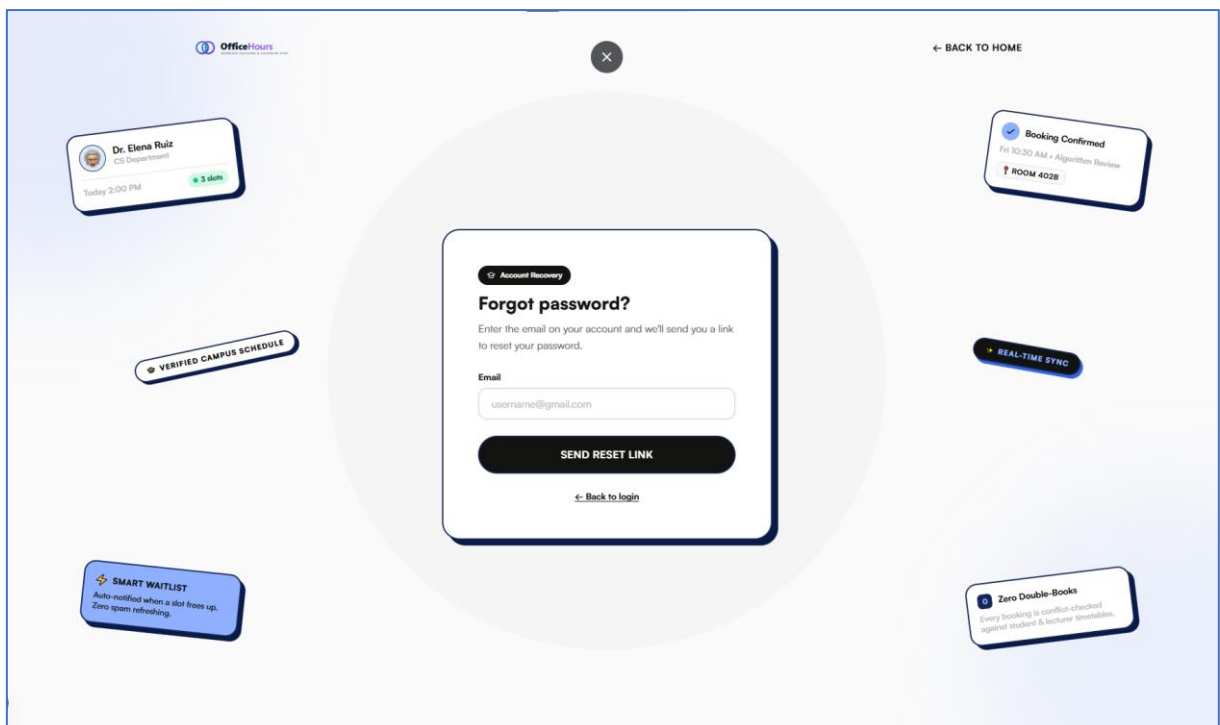


Figure 17. Password Recovery Flow

Shared Authenticated Workspaces:

- Role-Aware Dashboard Home: Transitions to a clean, high-density flat layout upon authentication. Renders customized views for Students (upcoming bookings), Lecturers (pending approvals), and Admins (system metrics).

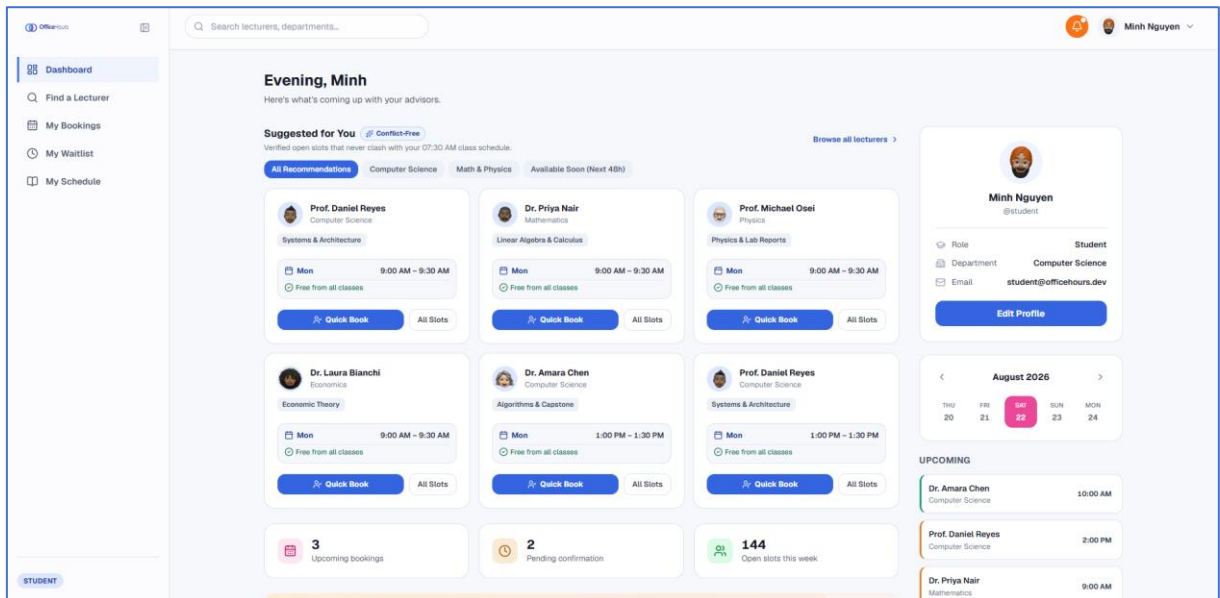


Figure 18. Role-Aware Dashboard Home

- My Profile & Security: Allows users to update personal details and manage notification preferences, secured by secondary dark-glass confirmation dialogs (ConfirmModal).

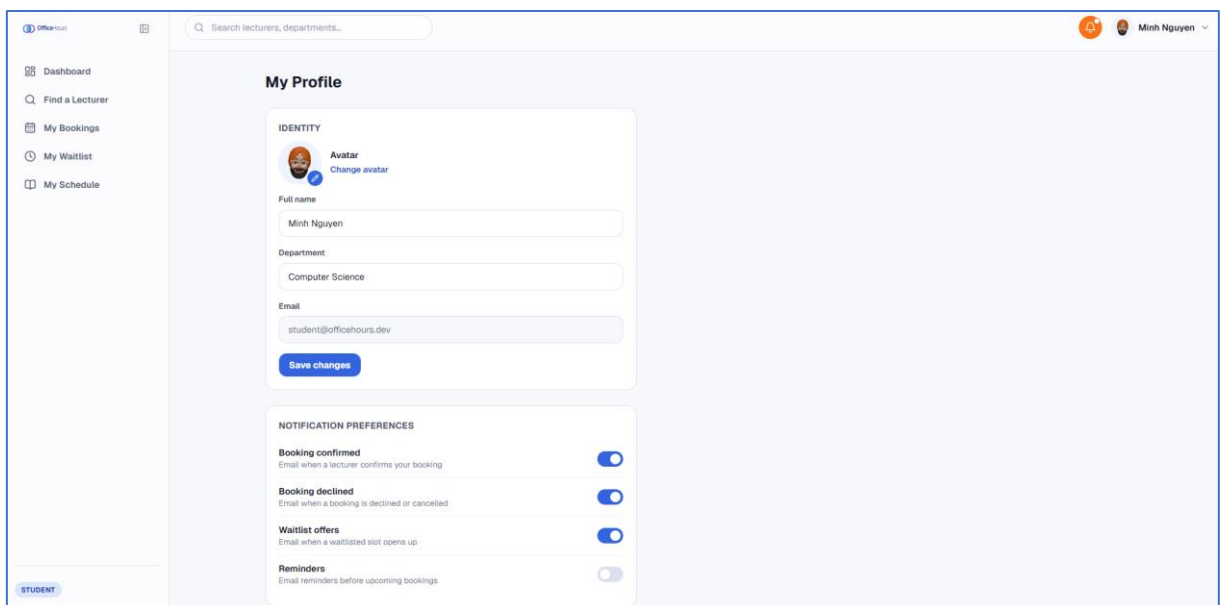


Figure 19. My Profile & Security

- Notification Center: Organizes systemic alerts into tabbed segments (Unread/Read), maintaining client state readiness for real-time notification streams.

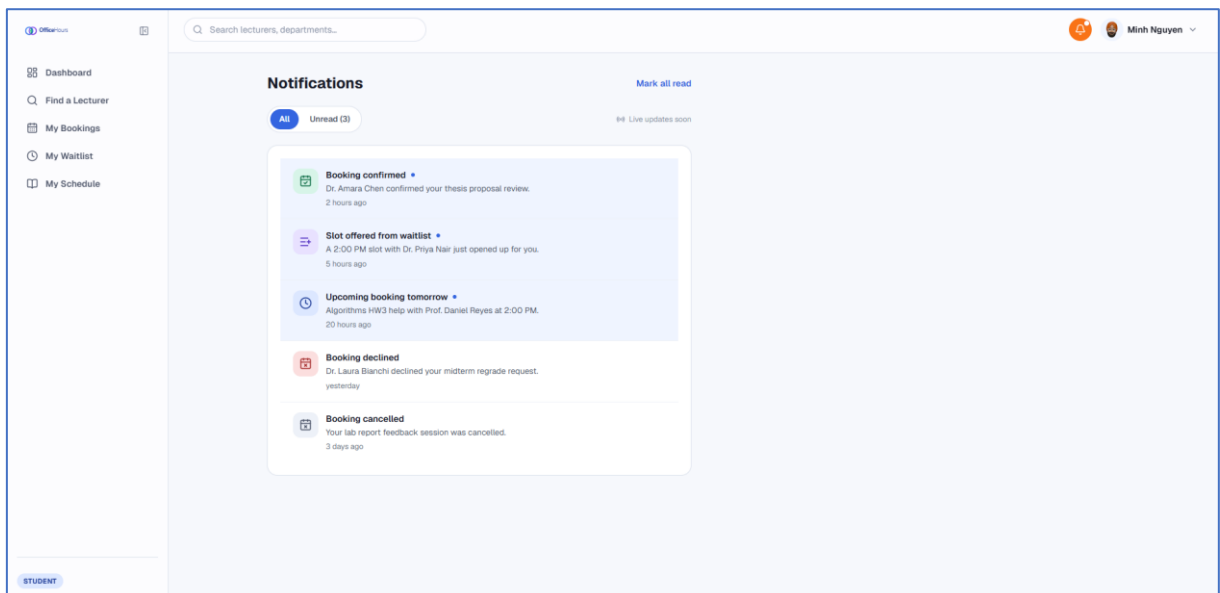


Figure 20. Notification Center

- My Bookings & Timeline View: Features a bidirectional sortable data table with tabular numeric fonts (tabular-nums) for timestamp alignment, paired with a detailed lifecycle timeline (PENDING → CONFIRMED → COMPLETED).

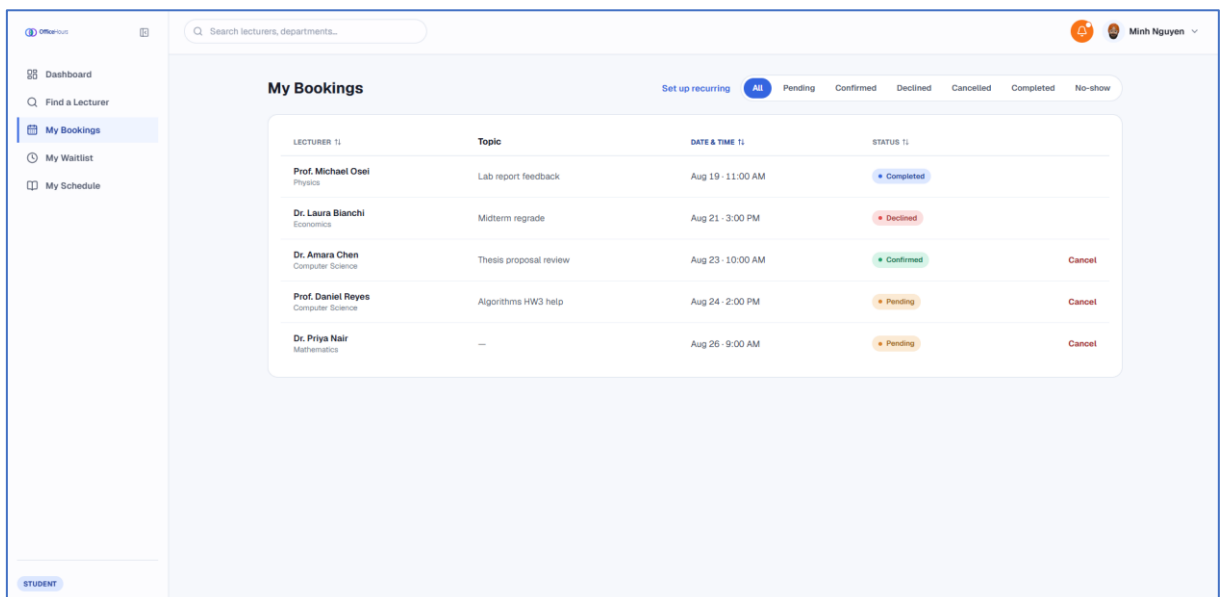


Figure 21. My Bookings & Timeline View

Student Operations Views:

- Lecturer Search & Directory: Offers a searchable directory with debounced text filtering to query faculty members by name or department.

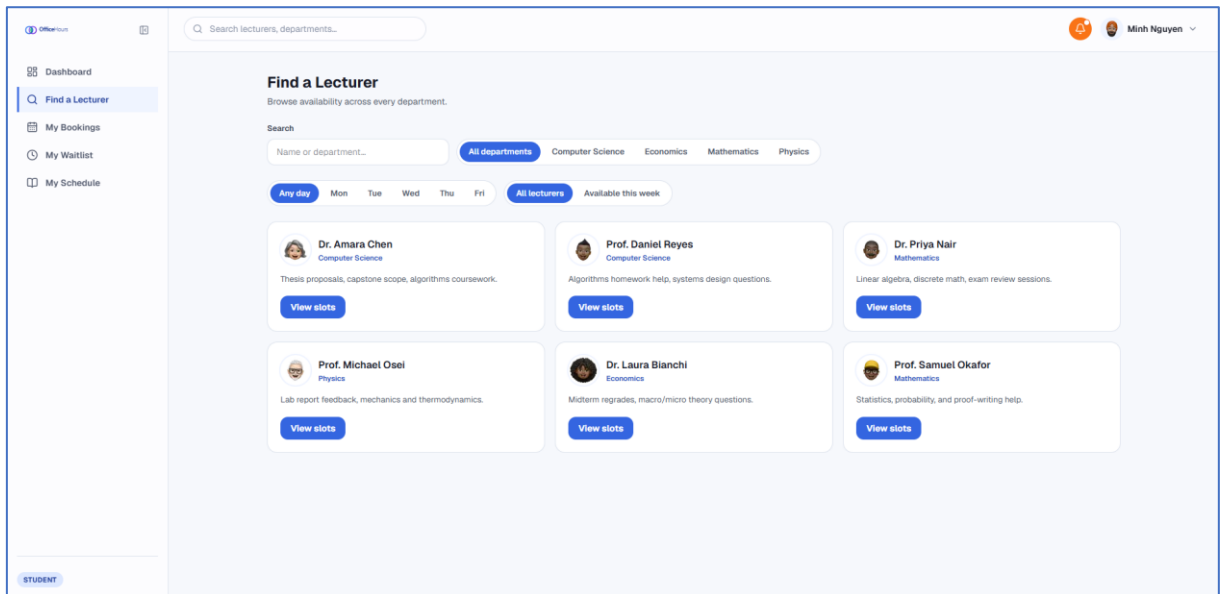


Figure 22. Lecturer Search & Directory

- My Class Schedule: The student's own imported timetable, rendered as a dual Timetable Matrix / Agenda List view with Mon-Fri / Mon-Sat / full-week and shift filters, alongside manual add/delete of individual class blocks.

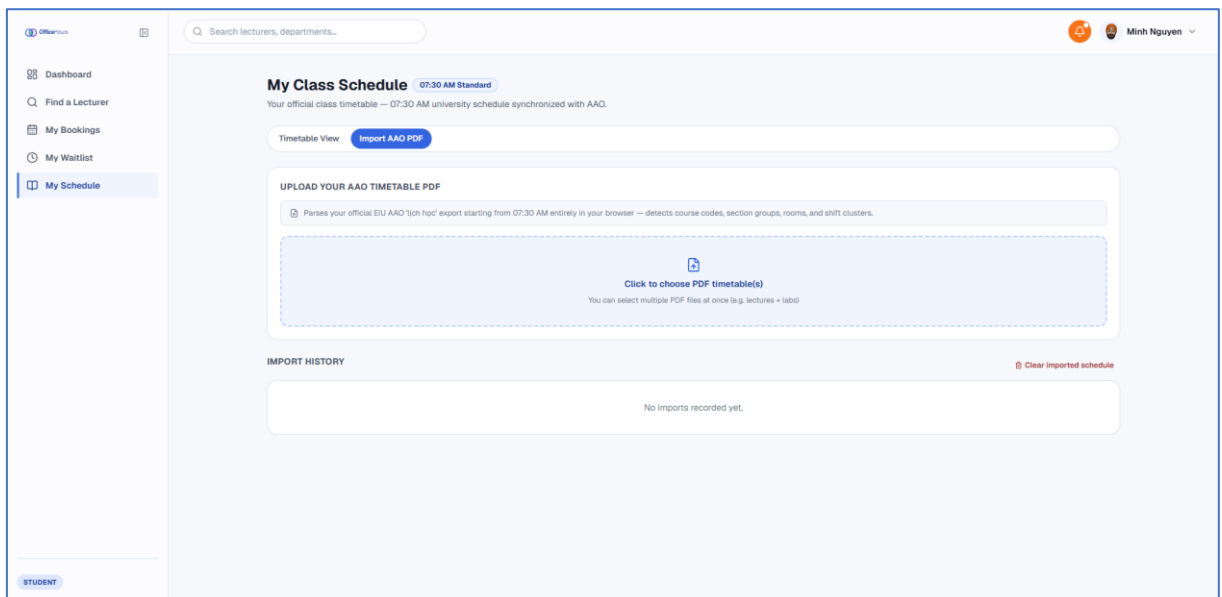


Figure 23. Import AAO PDF Dashboard

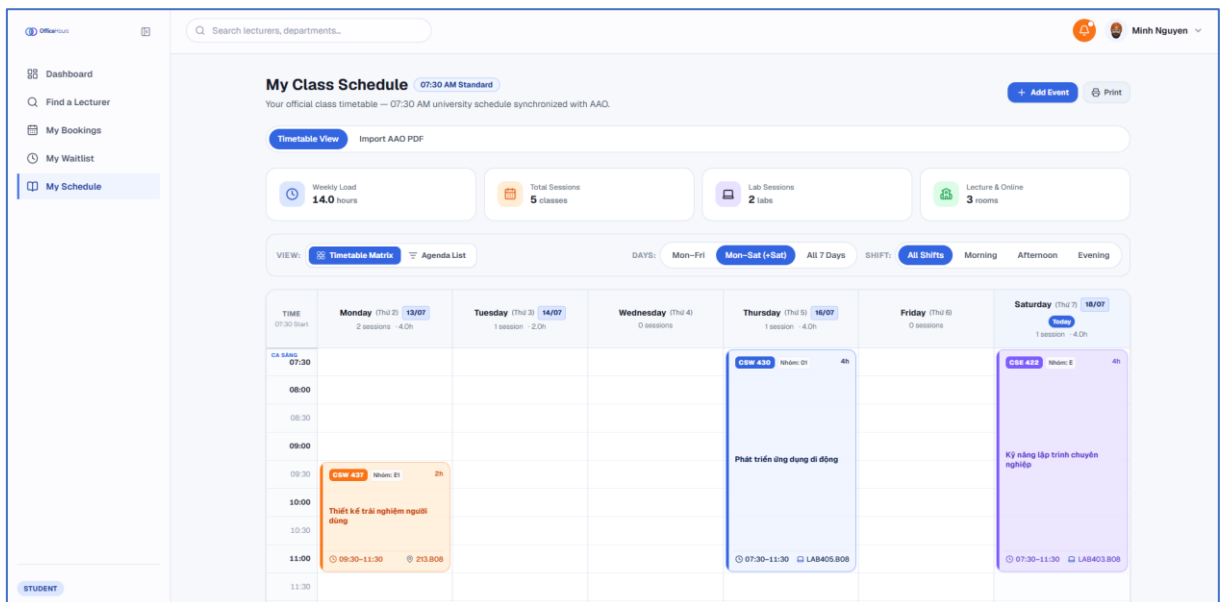


Figure 24. Time Table View

- Weekly Slot Picker: Displays consultation windows on a weekly columnar grid, automatically cross-referencing and disabling slots that conflict with the student's personal timetable.

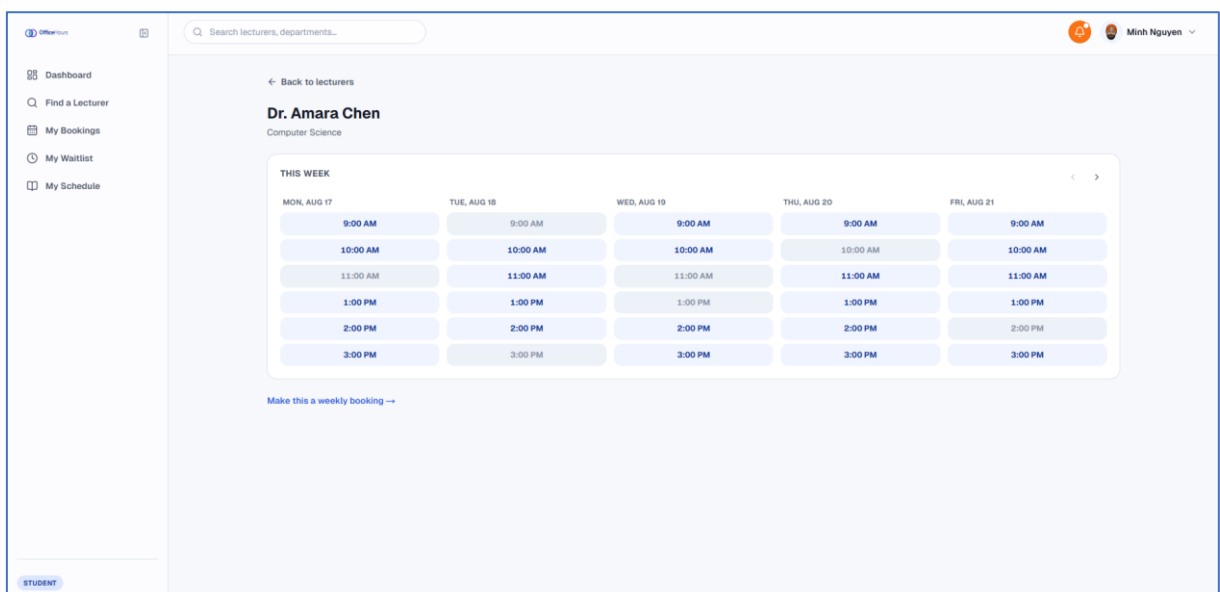


Figure 25. Weekly Slot Picker

- Group Booking & Recurring Setup: Supports peer email invitations inside a modal dialog and calculates term-wide recurring appointment offset dates dynamically.

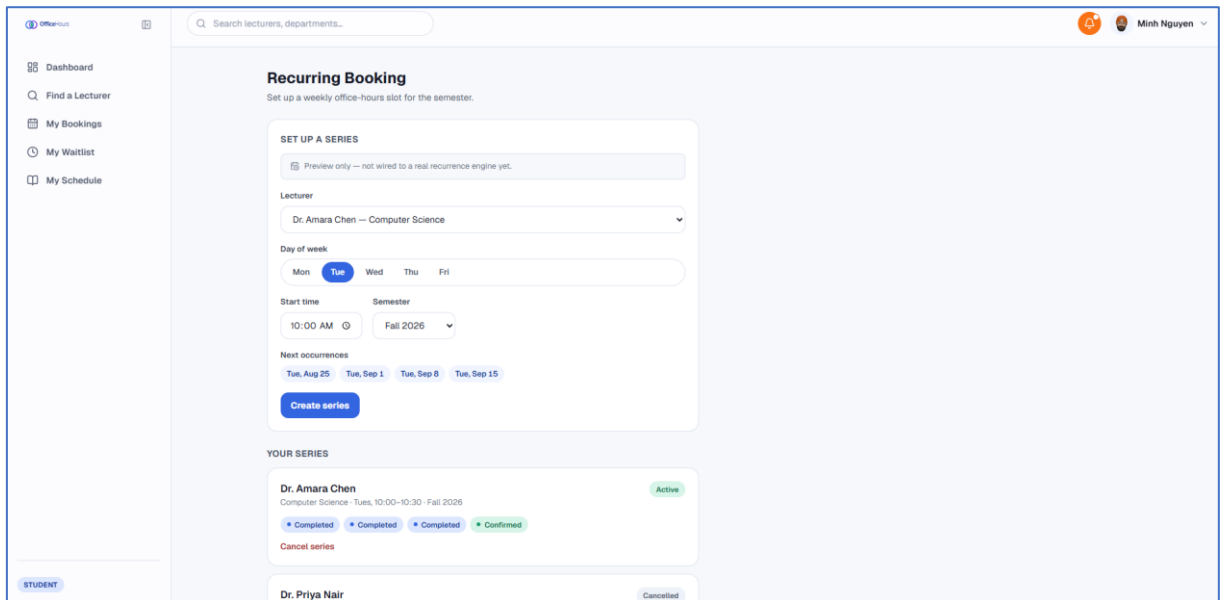


Figure 26. Group Booking & Recurring Setup

- Waitlist Management Dashboard: Displays active waitlist queue positions alongside countdown timers for newly offered open slots.

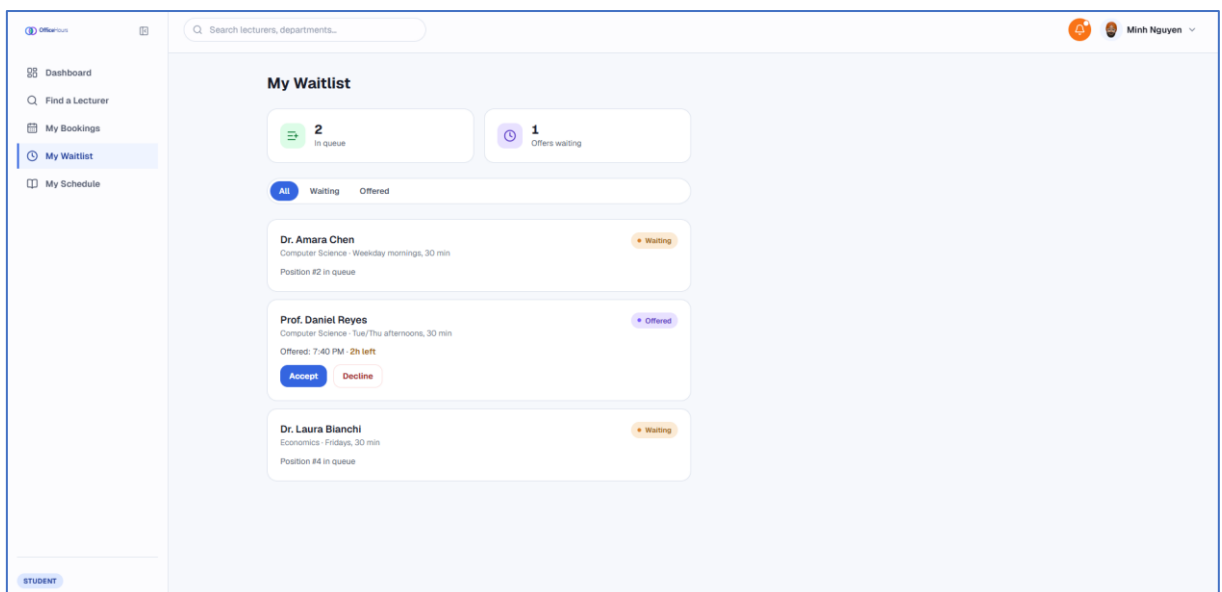


Figure 27. Waitlist Management Dashboard

Lecturer Operations Views:

- Availability Rules Manager: Provides a tabbed editor to define weekly recurring office hours alongside single-day exception overrides (e.g., conference blocks).

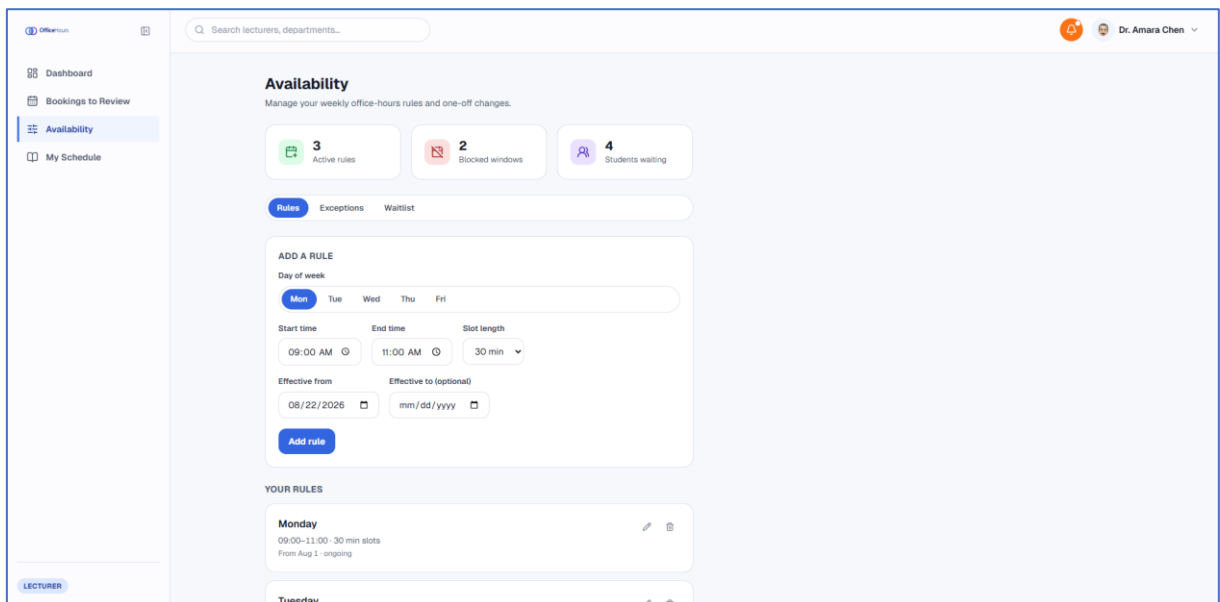


Figure 28. Availability Rules Manager

- Slot Waitlist Queue: Displays student waitlist rankings and countdown-to-expiry for each of the lecturer's individual consultation slots, on the Availability page.

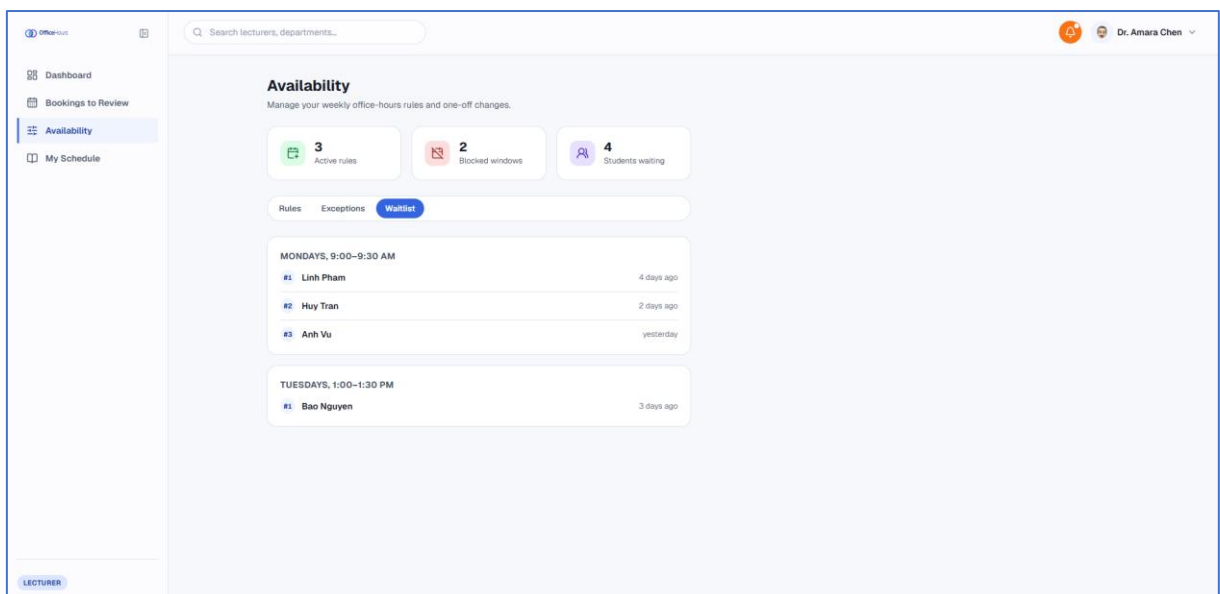


Figure 29. Slot Waitlist Queue

- My Teaching Schedule: A dedicated page mirroring the student schedule view - lecturers scan their own AAO teaching timetable PDF or add/delete class blocks manually, browsing the result as a Timetable Matrix or Agenda List with Mon-Fri / Mon-Sat / full-week and shift filters.

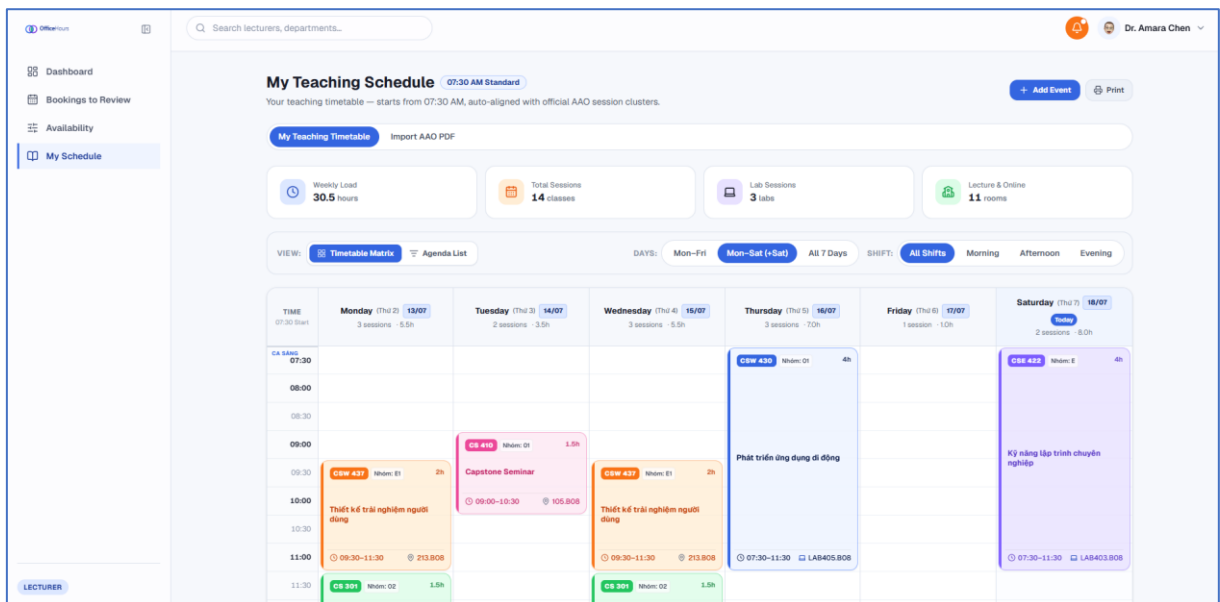


Figure 30. My Teaching Schedule

- Booking Review & Attendance Journal: Enables bulk confirmation/rejection of booking requests and post-appointment attendance/notes logging.

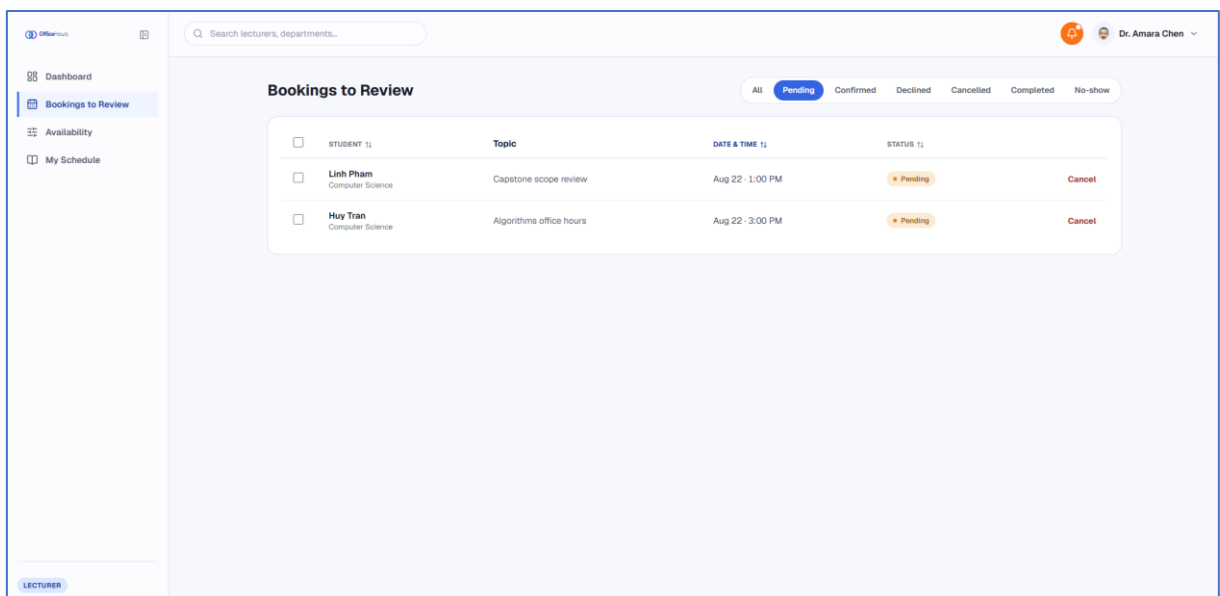


Figure 31. Booking Review & Attendance Journal

Administrative & Research Analytics Views:

- User & Semester Management: Supports inline updates to department affiliations and global operational semester boundaries.

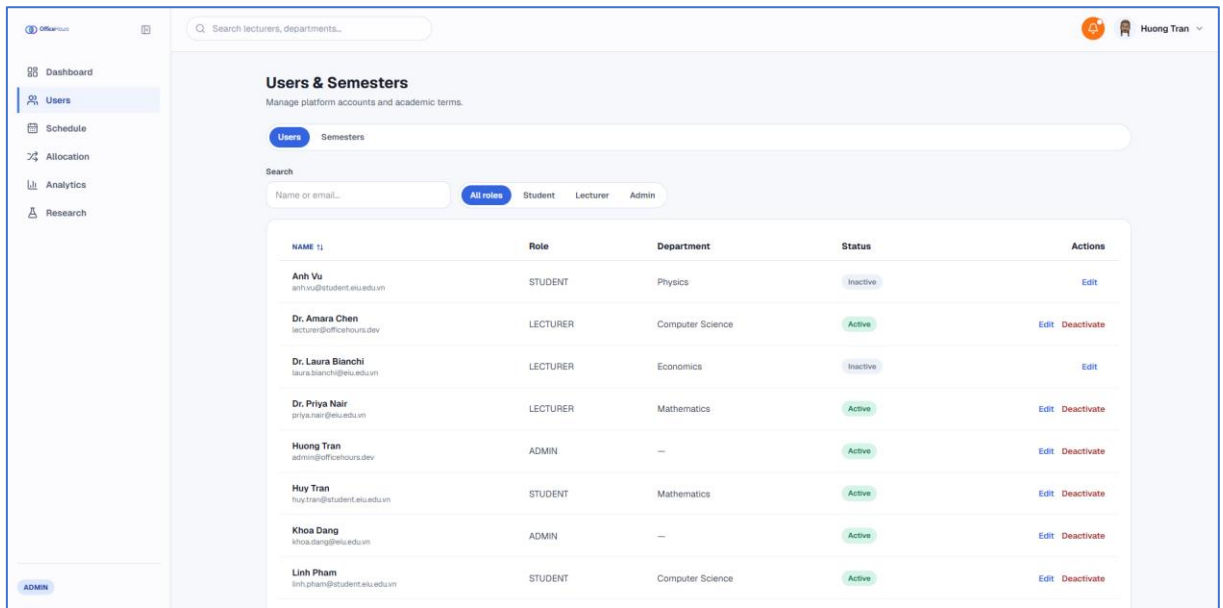


Figure 32. User & Semester Management

- PDF Timetable Importer: The same pdf.js-based scanner Students and Lecturers use on their own timetables, exposed as an admin support fallback - an owner picker (Lecturer/Student role toggle plus name search) selects whose schedule is being edited, then an uploaded PDF or the Manual Entry tab writes busy blocks into that person's timetable; a separate Slot Search tab browses open slots across all lecturers.

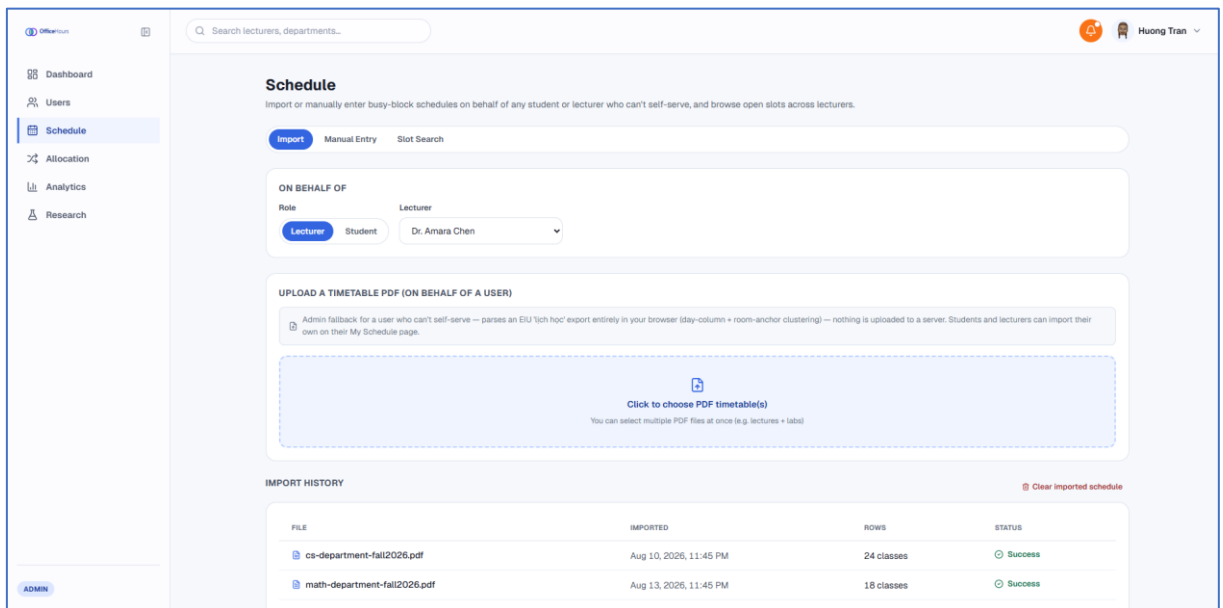


Figure 33. PDF Timetable Importer

- Allocation Policy Manager & Audit Log: Enables selection of waitlist algorithms (FCFS, NEED, HYBRID), displaying detailed audit logs with manual override capabilities (OVERRIDDEN).

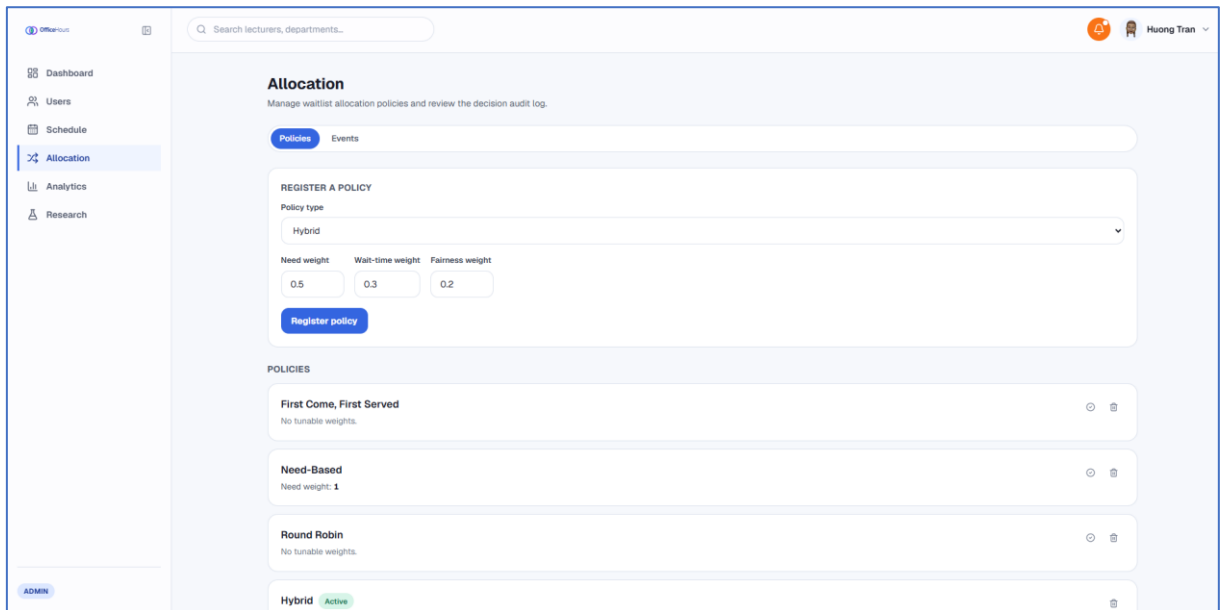


Figure 34. Allocation Policy Manager & Audit Log

- Analytics & Policy Benchmark Runner: Visualizes advisement workloads, Lorenz access curves, Gini coefficient metrics, and simulated efficiency-versus-fairness frontier scatter charts via Recharts.

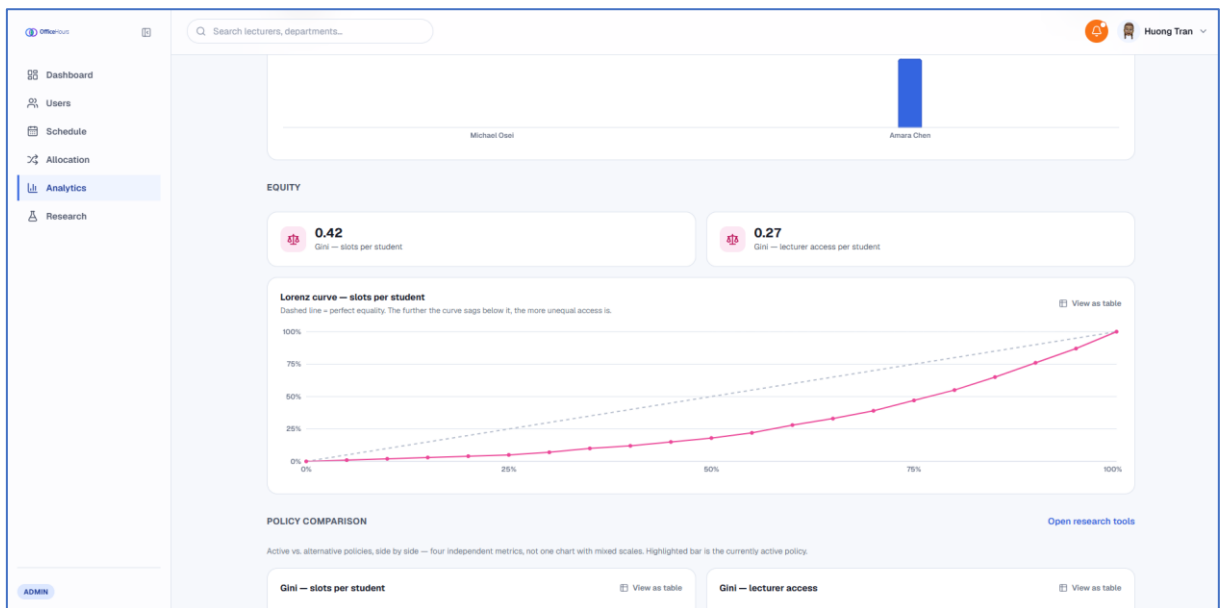


Figure 35. Analytics & Policy Benchmark Runner

4.2.2. Frontend Evaluation and Discussion

The evaluation of the web client focused on user interface responsiveness, client-side data state handling, and layout performance across various viewport sizes.

Layout Responsiveness and Render Performance:

- Isolating graphic-intensive CSS styles (such as backdrop blur filters) strictly to authentication screens prevented layout stutter on high-density data dashboards. Table components utilizing CSS tabular-nums maintained typographic alignment across

large tabular datasets. Debounced input handling on search fields eliminated unnecessary re-renders during real-time filtering operations.

Client-Side Schedule Conflict Prevention:

- The client-side schedule evaluator successfully verified timetable conflicts prior to form submission. By evaluating student time commitments directly within the browser render cycle, overlapping consultation slots were visually disabled, providing immediate feedback and reducing user input errors.

Algorithmic Visualization and Fairness Metrics:

- The analytics module successfully demonstrated client-side mathematical evaluation of scheduling equity using the Gini coefficient formula:

$$G = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n^2 \bar{x}}$$

- Through simulated demand loads, the frontend dashboard successfully rendered comparative trade-off curves between First-Come-First-Served (FCFS) and Hybrid allocation policies, confirming that the user interface can effectively synthesize and present complex decision-support metrics to administrative users.

CHAPTER 5. CONCLUSION AND FUTURE WORKS

5.1. Conclusion

Developing the frontend web application for the Academic Consultation and Office Hours Management Platform provided extensive practical experience in modern web engineering and user interface design. This project guided the development process through the software engineering lifecycle – from initial concept formulation and requirement analysis to interface design, component implementation, and client-side usability testing. It served as an opportunity to translate theoretical concepts of resource allocation into a responsive web application while building proficiency with essential development tooling, including Git for version control, Next.js/React, and Tailwind CSS for modular UI styling.

Building this system provided deep insights into designing user-centered software for academic environments. The platform was structured to simplify consultation scheduling, faculty availability tracking, and waitlist management across departments. Key features – such as role-aware dashboards (Student, Lecturer, Admin), interactive weekly time-slot pickers, and real-time visualization of access equity through Gini coefficients and Lorenz curves – demonstrated how well-crafted user interfaces can streamline administrative tasks and improve operational transparency. This experience also highlighted key areas for future growth, including server-side persistence, automated schedule synchronization, and predictive workload management.

Beyond technical implementation, the project significantly developed technical problem-solving and critical thinking skills. Overcoming development challenges – such as handling complex client-side state transitions, parsing timetable documents via pdf.js, and rendering interactive algorithmic trade-off charts using Recharts – required continuous research, experimentation, and debugging. Additionally, managing project milestones, maintaining clean code architecture, and documenting design decisions reinforced self-directed learning and structured engineering discipline, equipping me with essential skills for future software development projects.

5.2. Future Works

The web application currently establishes a comprehensive frontend foundation, offering core interfaces for role-based appointment scheduling, waitlist queue management, and allocation policy visualization. To evolve the platform from a client-side prototype into a fully operational, enterprise-grade academic ecosystem, several critical developments are planned for future iterations.

5.2.1. Backend API & Persistence Layer Implementation

The immediate priority for future work is the design and implementation of a robust backend architecture. Developing RESTful APIs using ASP.NET Core or Node.js, backed by a relational database system (such as PostgreSQL or SQL Server), will replace current client-side mock data handlers (*getMockOfficeHours*). This layer will enforce secure JWT authentication, manage transactional data persistence, and execute storage-level constraints to prevent double-booking at the database level.

5.2.2. Mobile Application Development

To improve accessibility for users on the go, dedicated cross-platform mobile applications will be developed using React Native or Flutter. The mobile clients will allow students and faculty members to receive push notifications for booking confirmations, check upcoming schedules on mobile calendars, and scan QR codes for fast attendance logging during consultation sessions.

5.2.3. Student Information System (SIS) API Integration

Connecting the system directly with institutional SIS and LMS will automate schedule synchronization. Live bidirectional API integration will replace manual PDF or CSV timetable imports, automatically detect course schedule changes and update consultation availability without administrative intervention.

5.2.4. Real-time Communication & Notification Engine

Implementing WebSockets or SSE will upgrade the notification infrastructure into a real-time communication pipeline. Live updates will immediately notify waitlisted students when a consultation slot opens up, broadcast schedule changes, and support real-time messaging between students and advisors during appointment setup.

5.2.5. Predictive Machine Learning for Demand Forecasting

Integrating predictive machine learning models will enhance administrative decision-making. By analyzing historical booking trends, the system will forecast peak consultation demand around midterm and final examination periods, enabling faculty and university administrators to proactively adjust office hour availability before scheduling bottlenecks occur.

These planned enhancements will transform the system into an intelligent, fully integrated, and scalable academic scheduling platform capable of supporting complex institutional workflows.

REFERENCES

- [1] Vercel, "Next.js Documentation", Next.js, 2026. [Online]. Available: <https://nextjs.org/docs>. [Accessed: Aug, 18, 2026]
- [2] Meta Platforms, Inc., "React Documentation," *React*, 2026. [Online]. Available: <https://react.dev>. [Accessed: Aug, 18, 2026].
- [3] Tailwind Labs Inc., "Tailwind CSS Documentation," *Tailwind CSS*, 2026. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: Aug, 18, 2026].
- [4] Framer B.V., "Framer Motion Documentation," *Framer*, 2026. [Online]. Available: <https://motion.dev>. [Accessed: Aug, 18, 2026].
- [5] Lucide Contributors, "Lucide Icons Documentation," *Lucide*, 2026. [Online]. Available: <https://lucide.dev>. [Accessed: Aug, 18, 2026].
- [6] Recharts Community, "Recharts Documentation," *Recharts*, 2026. [Online]. Available: <https://recharts.org>. [Accessed: Aug, 18, 2026].
- [7] P. Srinivas, "Rough.js Documentation," *Rough.js*, 2026. [Online]. Available: <https://roughjs.com>. [Accessed: Aug, 18, 2026].
- [8] Broadcom Inc., "Spring Boot Reference Documentation," *Spring*, 2026. [Online]. Available: <https://spring.io/projects/spring-boot>. [Accessed: Aug, 18, 2026].
- [9] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," *PostgreSQL*, 2026. [Online]. Available: <https://www.postgresql.org/docs>. [Accessed: Aug, 18, 2026].
- [10] GitHub, Inc., "GitHub Docs," *GitHub*, 2026. [Online]. Available: <https://docs.github.com>. [Accessed: Aug, 18, 2026].